

ParallelPilot: Supporting Coordination and Monitoring in Parallel AI Coding

TAO LONG, Microsoft Research AI Frontiers, Columbia University, USA

WEILI SHI, Microsoft Research AI Frontiers, USA

HUSSEIN MOZANNAR, Microsoft Research AI Frontiers, USA

MAYA MURAD, Microsoft Research AI Frontiers, USA

RAFAH HOSN, Microsoft Research AI Frontiers, USA

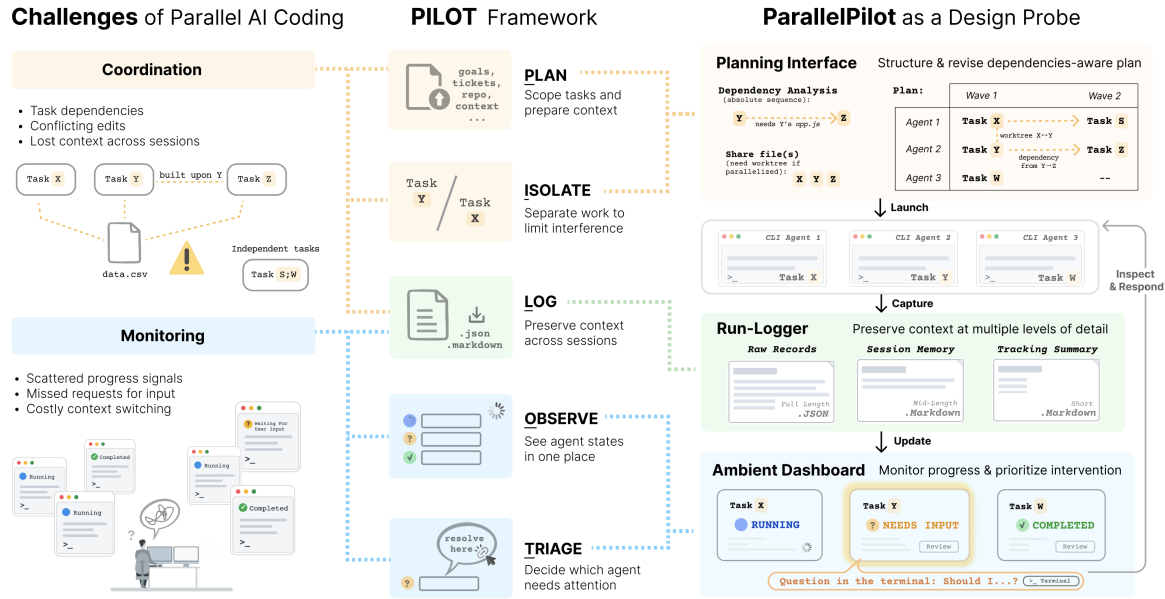


Fig. 1. Overview of two challenges in parallel AI coding—coordination and monitoring (left)—the PILOT framework (center), and the ParallelPilot design probe (right). PILOT links five supervisory practices (Plan, Isolate, Log, Observe, Triage) to interface support structuring parallel work, preserving session context, and directing attention to agents that need input. ParallelPilot implements these practices through a planning interface, a run-logger, and an ambient dashboard, alongside existing coding tools.

As coding assistants become increasingly autonomous, developers run multiple sessions in parallel, shifting the challenge from code generation alone to coordinating and monitoring concurrent agent work. Through a formative study ($N = 14$), we identified PILOT: five supervisory practices for Planning, Isolating, Logging, Observing, and Triaging parallel sessions. We present ParallelPilot, a design probe that instantiates PILOT through a planning interface, a run-logger, and an ambient dashboard alongside existing coding tools. In a counterbalanced within-subjects study ($N = 16$), ParallelPilot significantly increased ticket throughput by 63% in our short coding tasks and perceived efficiency by 1.69 points on a 7-point scale. It also clarified plans, dependencies, and intervention cues while reducing tracking effort and context switching. These gains were not accompanied by significant improvements in perceived control or perceived success in redirecting the agents. Our findings demonstrate the value of explicit supervision support and highlight a complementary design need: connecting high-level progress awareness with the implementation-level evidence developers need to evaluate and redirect agent work.

CCS Concepts: • **Human-centered computing** → **Human computer interaction (HCI)**; *Interactive systems and tools*; *Empirical studies in HCI*; User interface management systems; Interaction paradigms; Graphical user interfaces; Command line interfaces; Collaborative interaction.

Additional Key Words and Phrases: AI coding assistants, parallel AI coding, parallel programming, human-AI interaction, developer tools, multi-agent systems, oversight, awareness, supervision, context switching

1 Introduction

Software development is undergoing a fundamental shift—but not just in what code gets written, or how fast [15]. AI coding assistants are evolving from autocomplete tools into autonomous agents capable of executing complete engineering tasks, including writing code, running tests, debugging failures, and proposing architectural changes [42, 43]. As these agents become more capable, developers are increasingly experimenting with concurrent workflows, running multiple agent sessions on different tasks within and across projects [18, 24, 30].

We refer to this emerging practice as *parallel AI coding*: running multiple independent AI coding sessions concurrently, typically as separate agent instances or windows. Developers use this approach to make progress on multiple independently actionable tasks or projects at the same time, rather than working through them sequentially or repeatedly switching between them [7, 35, 44]. For example, a full-stack developer might set up three separate Claude Code sessions through the command-line interface (CLI) to handle three tickets: a frontend feature, a backend bug, and test writing. The developer monitors and coordinates all three concurrently, responding when the backend bug session asks for clarification while the others continue working. When the frontend session finishes, they review its result and send a follow-up to add a dark theme feature.

Parallel AI coding does not simply add more agents to the development process—it adds a new layer of coordination around them, creating substantial overhead [12, 35]. Developers must maintain a mental model of the parallel effort and respond to the events produced by each session. They need to decide how to divide work across sessions, what context each should receive, and which tasks must be completed before, after, or independently of one another. As work unfolds, they must also keep track of which session or terminal window corresponds to which task, how each task is progressing, and where intervention is needed. This becomes increasingly difficult as sessions in existing coding interfaces generate continuous streams of logs, code changes, errors, and input requests. Developers are thus left to piece together the state of their growing fleet across terminals, windows, and logs.

In this paper, we focus on two intertwined forms of supervisory work in parallel AI coding, adapting concepts from traditional supervisory control theory [37, 38]. *Coordination* involves establishing and maintaining the structure of parallel work, including task decomposition, dependencies, file boundaries, and agent assignments. *Monitoring* involves tracking live agent activity, detecting completions, errors, stalls, and input requests, and determining which sessions require intervention. We study these demands through their observable costs, including context switching, monitoring effort, and the need to recover or reconstruct session state after interruptions.

Through a formative study with 14 developers who regularly use parallel AI coding, we found that they already experience this dual coordination and monitoring burden. Most of them spent considerable effort deciding how to coordinate work across sessions and isolate them to avoid conflicts. To monitor the sessions, they developed ad hoc workflows combining terminal windows, markdown logs, custom dashboards, multi-monitor layouts, and other tools to track distributed agent state and surface issues. Yet these workarounds remained brittle and exhausting: developers missed when agents stalled or required input, lost track of which session was working on which feature, and struggled

to recover context after interruptions. Across these practices, we identified five recurring strategies—Planning, Isolating, Logging, Observing, and Triaging—which we synthesize as the PILOT framework.

We introduce ParallelPilot, a supervision cockpit designed to reduce the overhead of parallel AI coding. Rather than replacing developers’ existing agent CLIs, editors, or workspace arrangements, ParallelPilot adds a lightweight coordination and monitoring layer around them. Its three components correspond to complementary aspects of PILOT: (1) a planning interface that supports dependency-aware task structuring, (2) a run-logger that captures per-agent events and traces, and (3) an ambient dashboard that synthesizes this information into a glanceable overview, surfaces sessions requiring attention, and provides summaries for rapid context recovery.

We evaluated ParallelPilot in a counterbalanced within-subjects study with 16 developers, comparing matched coding tasks with and without the tool. ParallelPilot improved objective task performance: participants completed more tickets, reached full completion more often, and achieved higher throughput. Participants also reported improved perceived efficiency, better planning ability, situational awareness, and ease of context recovery while comfortably supervising more concurrent sessions. However, these gains did not extend to perceived control or participants’ perceived success in redirecting the agent through their interventions. Participants could keep up with parallel agent activity more effectively without necessarily feeling more confident in verifying or intervening on agent outputs. This highlights an important boundary of supervisory interfaces for parallel AI coding: making parallel work easier to coordinate and monitor does not, by itself, make agent work easier to verify or to intervene on.

Overall, we make three contributions:

- **The PILOT Framework**—an empirically grounded framework for parallel AI coding supervision from a formative interview study ($N = 14$), identifying five pillars: Planning, Isolating, Logging, Observing, and Triaging. Together, these strategies help mitigate the costs of coordination and monitoring.
- **ParallelPilot**—a supervision cockpit that operationalizes PILOT through three components: dependency-aware planning, run logging, and an ambient dashboard for glanceable monitoring and actionable triaging.
- **Empirical insights from a controlled within-subjects study** ($N = 16$) providing evidence of higher measured ticket throughput and lower self-reported tracking and context-switching burden in short parallel coding tasks, while perceived control and intervention success did not significantly change. Interview findings motivate a distinction between high-level progress awareness and the implementation context needed for steering.

2 Related Work

2.1 Managing Concurrent Coding Tasks and Coordination Challenges

Managing concurrent streams of coding tasks promises greater efficiency in task completion [41]. Studies and reports show that a single developer can now achieve output that previously required multiple engineers: shipping 72 story points per day and 22 PRs per week [39]; using four AI agents to complete projects in half the time of a linear one-at-a-time development workflow [2]. Cursor reported that parallel AI coding enables doubling weekly PR throughput and compressing an 18-month migration into a single engineer running a fleet of agents [7]. However, managing concurrent tasks and AI tools imposes two major demands: effective coordination and continuous monitoring [35]. We discuss the coordination challenge below and the monitoring challenge in Section 2.2.

Concurrent work scales up the amount of information people must coordinate at once. Developers must maintain an understanding of each task’s dependencies, priorities, conflicts, progress, timeline, needed context, relationships to the larger project, and past or future activities. Coordination then operates on two levels: selecting an initial set of

tasks and iteratively adapting as newly available capacity emerges. Thus, it requires decomposing and sequencing work into manageable, parallelizable components when tasks are set together. In engineering, tools like GitHub and designs like Gantt charts help externalize this structure through issue trackers, branches, pull requests, project boards, kanban, continuous integration systems, and version control [41]. Previous agentic tools introduce structural orchestration and a separate orchestrator agent to assist with coordination [8, 32, 47]. A recent study [44] shows parallel AI coding tools with stronger dependency-finding and coordination design lead to a 14% improvement in pass rate, a 2.10× wall-clock speedup, and a 35% reduction in API cost. Distributed cognition [17, 19] also provides coordination scaffolding. Hutchins’ cockpit example [19] illustrates how pilots manage and coordinate concurrent threads of information—altitude, speed, and communications—with instruments and ground control through shared states, displays, and adaptive procedures. DoubleAgents, a recent HCI tool powered by parallel AI agents [25], adapts this framework and offloads the human coordinator’s coordination efforts by managing parallel agents, visualizing the progress of threads of work, and codifying users’ coordination strategies as future alignment materials. Beyond structure, Vasilescu et al. [41] identify context switching as the key cost of parallelization. The structural complexity and lack of traceability of concurrent threads create coordination barriers, as developers must mentally switch contexts and recall relevant background information to understand other threads when viewing them with fresh eyes [1, 41]. Developers now create developer journals to document tasks so they can reference tangible trace artifacts and context-switch effectively among sessions [33, 36]. However, as these AI systems become more autonomous and proactive, structuring and preparing work for parallel coding may require additional support.

2.2 The Monitoring Challenges of AI Coding Assistants: Legibility and Human Factors

AI coding assistants produce code, logs, and explanations at overwhelming volumes and speeds, creating a legibility problem for human monitoring [10, 14, 21, 34]. Anthropic reported that one single user task generates roughly 12 Claude actions and 3,200 words [16]. Generation is 5–7× faster than humans can read or comprehend [6]. A recent quantitative study [48] echoed that coding agents generate output at 33.9–46.8 tokens per second, compared with roughly 5.3 tokens per second¹ for average adult reading speed [3]. Beyond intermediate logs, final code generation is also overwhelmingly fast: AI agents average 140–200 lines of meaningful code per minute, whereas focused humans read 20–40 lines [6].

Even when coding assistants make their processes readable, developers may still not notice, read, or understand what the agent is doing. A prior HCI study [4] shows that developers frequently skim or disregard lengthy execution traces to avoid substantial cognitive demands and information overload. Other studies [29, 45] show that such skimming is common among developers who overrely on coding assistants, perceiving them as sufficiently competent or as a convenience. However, issues frequently arise when users miss critical intervention points due to insufficient monitoring [1, 5]. An empirical study of over 100 developers collaborating with coding agents [46] found that 94% of the developers failed to detect agent-inserted sabotage; even when explicit monitor warnings were present, 56% accepted malicious code, reporting that they “*didn’t dwell on*” the alerts. The identified failure modes are minimal code review (67%), plausible cover stories (22%), and overtrust built from routine use (11%). Situational awareness theory [20] emphasizes that the ability to detect and be aware of potential problem cases is a prerequisite for user control and intervention. The human-AI oversight community [22] has also noted that users often struggle with “*recognizing critical situations or AI behavior*” and “*executing timely interventions*”. Recent HCI efforts target this problem by building

¹Converted from the estimated average silent reading speed of 238 words per minute for adults reading English non-fiction [3], using the standard approximation of 1 token $\approx$ 0.75 English words [31].

structural visualizations of traces [9, 14] and centralized but tiered dashboards [10, 26] to help developers understand agent logs and status. Many developers adopt practices to constrain the intermediate logs and unnecessary explanation in output by adding an `AGENTS.md` file or instructions to save tokens and improve readability [28].

Running multiple parallel AI coding assistants compounds monitoring challenges—e.g., enabling user interaction without disrupting ongoing processes, and allowing prioritization or interruption across agents [35]. A six-week deployment of three concurrent research agents confirms that parallelizability amplifies the oversight burden [23]; the authors cite “overwhelming information” and users’ inability to manage it, prompting categorized traces of human, agent, and infrastructure events to aid sense-making. This motivates the need for support that helps developers track parallel agents, intervene selectively, and avoid overreliance on automation.

3 Formative Study: Understanding How Developers Supervise Parallel AI Coding

3.1 Overview

To characterize how experienced developers coordinate and monitor multiple concurrent AI coding sessions, we conducted a 60-minute formative interview study with 14 developers. We investigated their motivations for adopting parallel AI coding and their strategies for managing such a process. The formative study was guided by two RQs:

- RQ1: Why, when, and how often do developers turn to parallel AI coding?
- RQ2: What strategies and practices do developers use to coordinate and monitor parallel sessions?

3.2 Participants

We recruited 14 participants (P1–P14) through an internal mailing list for researchers and developers within our research organization. The recruitment message invited developers who regularly use AI coding assistants and have extensive experience running more than two AI coding sessions in parallel. The 14 participants (4 women, 10 men) spanned roles comprising 7 PhD research interns, 3 software engineers, 2 senior researchers, and 2 engineering managers. Seven participants were in the 25–34 age range, three in the 45–54 range, two in the 18–24 range, and one each in the 35–44 and 65+ ranges.

All participants were highly experienced coders: six participants reported more than 10 years, three reported 7–10 years, and five reported 4–6 years. All participants used AI coding assistants more than three times per day in their general workflow and demonstrated substantial seniority and expertise in parallel AI coding workflows—ten reported running multiple AI coding sessions in parallel more than 30 times, while four reported doing so 10–30 times.

3.3 Procedure

Each participant took part in a 60-minute semi-structured interview, either in-person or via Microsoft Teams. Before the interview, participants completed a short demographics and AI experience survey. During the interview, we asked participants to describe their current parallel AI coding workflows and invited them to screenshare and walk through a real project or setup in which they had used concurrent agents. We asked participants to map out the sessions running and what each one was working on. We probed how participants tracked each session: what they actively watched versus what they skimmed or ignored, what signaled that a session needed attention, how they judged a session’s output, and how they knew a session was done. See Appendices D and E for the survey and interview questions.

All participants provided informed consent to participate and to have the interview recorded for transcription and analysis. They were compensated 50 USD, and the study protocol was approved by the internal ethics committee.

3.4 Data Collection and Analysis

Our dataset comprised the survey responses, interview recordings and transcripts, and screen-shared walkthroughs described above. We analyzed survey responses descriptively to characterize participants' experience and parallel coding practices. We used inductive affinity diagramming [11, 27] across transcripts, notes, and screen-shared observations to identify recurring patterns in task decomposition, session organization, and monitoring. Codes and themes were refined through multiple rounds of feedback and discussion with all co-authors, informing a preliminary framework for supervising parallel coding workflows.

3.5 Findings

3.5.1 RQ1: Why, when, and how often do developers turn to parallel AI coding?

Summary: Participants used parallel AI coding to fill workflow downtime, such as waiting for builds or tests. Participants reported parallel AI accounted for nearly as much coding as single-session AI (45.3% vs. 48.6%). CLI interfaces dominated, Copilot and Claude/Claude Code were the most common tools, and three concurrent sessions were typical.

Current Coding Workflow Distribution and Setup. As shown in Figure 2, participants estimated how their current coding practice was distributed across three modes: AI-assisted coding was divided almost evenly between single-session AI ($M = 48.6\%$) and parallel multi-session AI ($M = 45.3\%$), with only 6.1% of coding occurring without AI.

CLI-based interfaces were the dominant setup for AI-assisted coding, mentioned by 13 of 14 participants (92.9%), followed by IDE-integrated chat interfaces and standalone web/desktop AI applications, each used by 11 participants (78.6%). In terms of tools, Copilot was the most ubiquitous, appearing across 11 participants (78.6%), followed by Claude or Claude Code (10 participants, 71.4%). ChatGPT was reported by 6 (42.9%), while Codex and Cursor were each reported by 4 (28.6%). P8, P3, and P7 were heavy users of parallel AI workflows, allocating 80%–100% of their coding time to parallel AI, primarily using Claude Code and Copilot CLI.

Working with three AI coding sessions at once was the most common setup, reported by 8 of 14 participants. Most participants worked with two to four concurrent AI coding sessions. The maximum number of sessions participants had ever run at once ranged from three to eight; five participants reported five, and one reported eight. One participant, P6, described a cloud-based workflow that allowed them to delegate small, isolated issues across 20 sessions.

Downtime as a Trigger for Parallel AI Coding. Beyond how often participants ran parallel sessions, our data point to a consistent trigger for *when* they did so: idle time in their workflow—periods when they were waiting for a single agent to complete. P9 stated, “*I don’t wanna wait sequentially for things to happen,*” while P6 noted, “*[now] I can actually work on multiple things at once [...when] I have downtime or I’m waiting for things.*”

Participants described parallel AI coding as transforming the familiar developer experience of waiting for tools to finish. P5 shared that “*parallel AI coding is useful to do while you’re waiting for something... having something else going is just nice.*” As illustrated in Figure 3a, P5 contrasted this with the classic developer trope of being idle while code compiles, referencing the xkcd comic “Compiling” (Figure 3b): “*It will no longer be like that [comic] with the still compiling... I can’t be like, ‘Oh, I don’t need to doomscroll. My code’s working for me.’ ... Now when I’m using Claude Opus, running five or six things at a time, [I’m] pretty much always answering questions from the machine or trying to redirect... so it really becomes like you’re doing full-time coding again.*” (P5) This illustrates how parallel AI coding converts idle waiting time into an active orchestration loop. P3 described this as an explicit scaling strategy: “*I feel I want to scale by running multiple concurrent sessions... trying to keep them and myself busy.*”

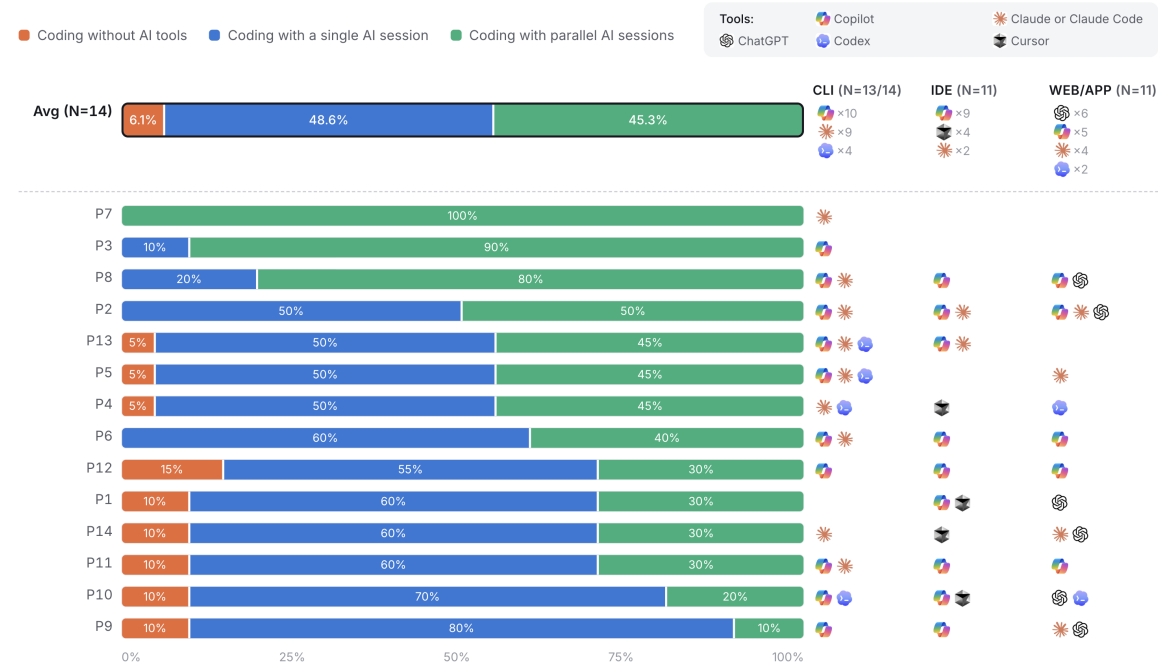


Fig. 2. Current coding workflow distribution self-reported by participants ($N = 14$). Each bar shows the percentage of coding time spent with no AI tools, a single AI session, or multiple AI sessions in parallel. Participants are sorted by parallel AI usage (highest to lowest); the group average is shown at the top, separated by a dashed line. Three additional columns on the right indicate which tools each participant uses across CLI, IDE-integrated chat interfaces, and WEB/APP (standalone web/desktop AI applications), with aggregate usage counts displayed in the average row.

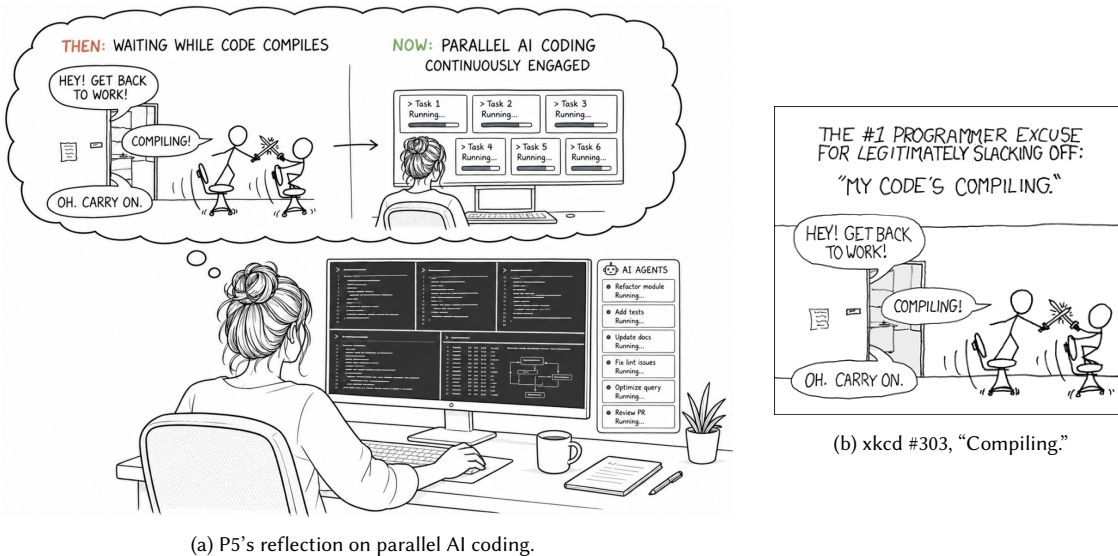


Fig. 3. P5 invoked the familiar developer trope depicted in xkcd #303 to describe how parallel AI coding changes the experience of waiting. Instead of disengaging while code compiles, developers remain actively engaged by monitoring, answering, and redirecting multiple concurrent AI sessions. Source: Randall Munroe, xkcd #303, <https://xkcd.com/303/>, licensed under CC BY-NC 2.5.

3.5.2 RQ2: How Do Developers Coordinate and Monitor Parallel AI Coding?

Summary: Parallel AI coding required active supervision: all 14 participants reported coordination and monitoring overhead. We introduce their management strategies here as PILOT; Section 4 develops each strategy with evidence.

Downtime made parallel AI coding possible, but participants emphasized that making it useful required active supervision. All 14 participants described a real coordination and monitoring tax. P2 put it plainly: “*after a while [running concurrent sessions]... if it’s this feature and that’s that feature, it’d be over your brain, fried quickly.*” The tax showed up as a recall cost too: P6 reported that they “*couldn’t even remember what I had told Copilot*” across sessions. P12 described feeling “*like a quarterback across three sessions,*” relying on peripheral glances and occasionally missing alerts when absorbed in one thread, calling the work “*a little more tiring.*” P7 found long logs “*overwhelming*” and asked agents for high-level markdown summaries instead, and P4 limited themselves to a single screen just to “*keep track.*”

Participants adapted their workflows to manage these demands, revealing five recurring strategies: **Planning** what should happen in parallel, **Isolating** sessions to prevent interference, **Logging** context to preserve continuity, **Observing** agent state without constant inspection, and **Triaging** situations requiring human intervention. We synthesize these strategies into the **PILOT** framework. The following section continues our answer to RQ2 by grounding each strategy in participants’ accounts and concrete workflow examples.

4 The PILOT Framework

As shown in Figure 4, the PILOT framework characterizes the strategies that help developers manage parallel AI coding: **Planning**, **Isolating**, **Logging**, **Observing**, and **Triaging**. As the number of concurrent agents scales up, developers must not only *coordinate* the work around them—deciding what can run in parallel, preventing sessions from interfering with one another, and preserving context across interactions—but also *monitor* ongoing work, maintain awareness of agent progress, and decide when and where to intervene.

We call the framework PILOT to evoke how airplane pilots coordinate and monitor through cockpit supports rather than track everything themselves, drawing on Hutchins’s account of distributed cognition [19]. As AI agents take on more of the direct programming work, developers increasingly pilot a portfolio of concurrent agent work threads. PILOT does not impose a fixed sequence: developers may move between and revisit earlier pillars as task and agent states change. Though derived from parallel AI coding, PILOT points to a broader problem of distributing limited human resources (time, attention, effort, etc.) across multiple concurrent activities by structuring and isolating work, externalizing context, maintaining situational awareness, and selectively allocating attention where needed.

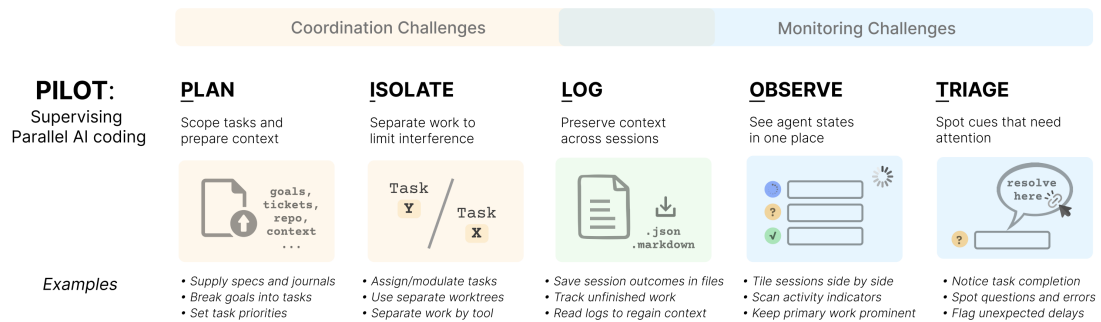


Fig. 4. The PILOT framework for supervising parallel AI coding. Five complementary practices address coordination and monitoring challenges: Planning, Isolating, Logging, Observing, and Triaging. Logging bridges both challenge areas by preserving context across sessions. Examples beneath each pillar illustrate developer practices; the pillars are not a fixed sequence.

4.1 P — Planning: Scoping and Preparing the Work

Once downtime created an opening for parallel work, participants did not immediately launch multiple coding sessions. Instead, they scoped the work, established priorities, and prepared the agent context and scaffolding.

Participants first identified a manageable set of tasks to run concurrently by breaking larger, ambiguous goals into smaller units. Participants commonly identified a primary task for closer attention alongside secondary tasks that could be checked later, giving them a tentative agenda for where to look first as multiple sessions progressed. Then, participants prepared the context that agents needed before launch. This included specifying preferred approaches, relevant files and folders, constraints, and success criteria. P2 described a repeatable handoff containing “*spec, prior state, plan, and work done*,” alongside a journal preserving all past decisions. Several participants provided tickets, PR links, and background notes to help agents quickly orient themselves (P3, P4, P6, P12). Nine participants developed reusable artifacts to improve consistency across sessions: six described encoding preferences into pipelines and markdown files for uploading, while three others built them as skills. P13 explained, “*most of my skills are for repetitive tasks...*” P2 used an issue-driven development `AGENTS.md`, while P11 added a “grill me” prompt to challenge and refine their plans.

Together, these practices show, rather than increasing concurrency by simply opening more sessions, participants deliberately prepared the work and its surrounding context so that parallel execution could remain manageable.

4.2 I — Isolating: Managing Separability

Once participants decided what to delegate, they had to keep parallel sessions from interfering with one another. P6 explained that parallelism became useful “*once a project has enough space for non-overlapping things.*”

To exercise separability, 11 of 14 participants modularized code and tasks, splitting work so that agents would not touch each other’s outputs. P7 looked for ideas “*not running to the same file*”, P13 ran “*three very parallel threads, which did not rely on each other*”, and P9 split along subtasks: “*if it’s a different subtask of the same project, I still have some kind of parallel workflow.*” Many participants also relied on version control primitives. P3 used worktrees and branches so that concurrent sessions would not “*mess with other people’s branches*”; P2 kept to one branch at a time, since “*multiple branches gets too confusing too quickly*”. Some participants even separated by tool use beyond code: P10 split work by using “*ChatGPT only for proofs, Codex to implement experiments*”, while P14 started a fresh conversation whenever pollution felt likely, “*I try less to pollute the whole conversation and make another session.*” The other three sat at opposite ends. P8 and P4 cautiously isolated at the whole-project level, afraid they would “*pollute each other’s context and make conflicting changes,*” whereas P11 skipped boundaries, accepting overlap and resolving merge conflicts later.

These practices show that successful parallelism depended on separability: tasks were independent enough that agents would not overwrite, duplicate, or conflict with each other. Misjudging it was costly: P2 found that overlapping work in the same repository cost “*more time later reconciling all of the differences*” than doing the work sequentially.

4.3 L — Logging: Externalizing Memory

Moving between multiple sessions made it difficult to remember what each agent had been asked to do, what it had tried, and why decisions were made, especially as AI histories and reasoning could quickly become inaccessible. P6 described this breakdown: “*I could [now] focus on the details of one [window] ... and then I couldn’t remember or even find what I had told Copilot to do in the first window by the time I got back to it.*”

To manage this burden, participants borrowed from traditional developer journaling, externalizing their memory and AI session history to persistent documents on local computers or in the cloud. Seven participants built markdown

or text files, logging decisions and outcomes of the sessions, so they could check for themselves and later feed them back into the next session as context. P4 used files such as NOTEBOOK.md and RESULTS.md; P2 kept per-issue journals; P7 maintained one markdown file per task with plans, attempts, progress, and remaining work. P11 described reading these documents to recover state: “*I read quite a bit of markdown files... I’m not looking at code.*”

These logs were not passive records but active coordination devices: a spec could remind the developer of scope, support later verification, and provide context for steering future sessions. By offloading session context to durable artifacts, developers reduce the working memory cost of switching between sessions, recover context faster after interruptions, and promote a sense of security.

4.4 O — Observing: Assembling a Centralized View

After launching parallel agents with logging in the backend, participants built makeshift control rooms out of terminals and side panels across one or two extended monitors alongside their laptop screen to observe raw agent states.

While screen setups varied, ten of the 14 participants arranged every active session so that a single glance could reach it—an active-monitoring full screen, requiring no active switch to check. Five of them split their main screen to keep two to four parallel sessions visible side by side, catching most status changes by scanning across them, though the windows shrank enough that fine detail was easy to miss. Two CLI-only users stacked terminal windows into one view on their main screen and watched the tab spinner in the navigation bar to determine their status. Three ran an IDE alongside a stacked chat or CLI panel in their main screen, keeping a full-detail primary session identified during *planning* apart from peripheral ones they could absorb without switching windows. The remaining four gave each agent its own full screen, swapping screens entirely to check on an agent or closing one agent window to open another—a deliberate act that marked a task switch. Though built around each user’s habits, they all shared the same cost: turning away from the current task, even briefly, often led to missed agent updates.

Across all participants, we see clear benefits of centralization: a single point of contact for every agent—one place to both check status and intervene when needed. Centralization lowers the manual cost of switching between screens and focuses user attention and potential steering more effectively.

4.5 T — Triaging: Prioritizing Intervention Cues

Even with a centralized view, participants found it difficult to determine which session needed attention now. Important moments could still be buried in long logs and scattered across windows. Triage therefore involved continuously detecting which session had become actionable and deciding what required intervention. In participants’ current practice, triage often relied on lightweight, improvised observation. Participants turned their heads or clicked between sessions, watched for spinners or activity cues (e.g., the Claude logo or Copilot’s blinking dot) to stop, and periodically discovered that an agent had been waiting for a question no one had answered. For longer-running or less observable tasks, they constructed additional signals, such as checking log statistics, generated files, dashboards, or verification outputs at intermediate points. Some even built lightweight visualizations to make otherwise difficult-to-interpret numbers easier to assess. These practices helped expose specific triggers, but were often indirect, manual, and easy to miss. P3 and P6 linked delayed detection to lost productivity, with P3 describing it as “*wasting my agent’s time,*” suggesting a need to surface actionable changes sooner.

Across tasks, participants identified four signals that most often triggered intervention. A *completion* signal indicated that a session was ready to move to the next task. *Requests for clarification* and *errors or bugs* required immediate intervention, yet were also the signals participants most reliably missed: “*when I come across [an]other session it’s*

stuck there” (P1). Finally, an *unexpected delay*—such as a deployment that should have taken two or three minutes but continued past ten—prompted participants to inspect the session and determine whether intervention was necessary.

Thus, triage involved turning concurrent activity into an actionable queue: identifying or constructing signals of completion, blockage, or abnormal progress, then prioritizing interventions.

5 ParallelPilot: A Design Probe for the PILOT Framework

We built ParallelPilot as a design probe that instantiates the PILOT framework. It is a lightweight web application that runs alongside developers’ existing coding CLIs, editors, and workspaces, supporting the *coordination* and *monitoring* of parallel AI coding. There are three integrated components: a planning interface that externalizes dependencies and structures execution (**Plan, Isolate**); a run-logger that connects agent sessions to tasks and preserves context (**Log**); and an ambient dashboard that makes agent status glanceable and highlights intervention cues (**Observe, Triage**). Figure 5 maps the three components to the overall workflow, with interface thumbnails providing visual orientation. The walkthrough below presents enlarged views of planning and triage (Figures 6 and 7).

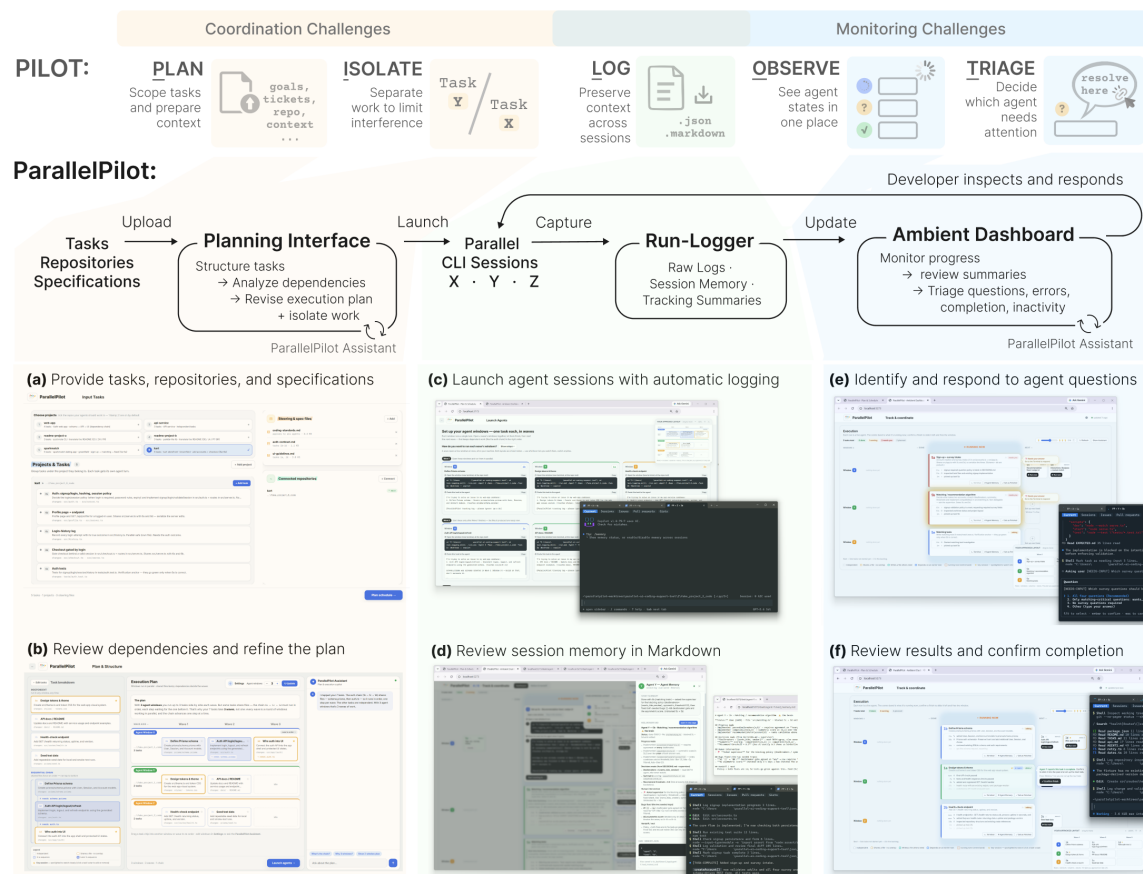


Fig. 5. ParallelPilot instantiates PILOT through three integrated components: a planning interface for task preparation and dependency-aware scheduling (a–b), a run-logger for automatic capture and session memory (c–d), and an ambient dashboard for monitoring progress and triaging agent questions and completion (e–f). The interface thumbnails map these components to successive stages of the workflow; enlarged views of (b) and (e) appear in the system walkthrough.

5.1 System Design

We illustrate the design through Amelia, a mid-level software developer persona who has tried parallel AI coding but found it difficult to track overlapping edits and recover context between sessions. She is working on two projects: 1) an internal support portal requiring **1a** Fix ticket pagination, **1b** Add CSV export, and an **1c** improved empty-state component; and 2) a separate data-import CLI requiring **2a** Add CLI retry handling and **2b** Add retry regression tests. The walkthrough follows her from planning and launching agent sessions to monitoring progress, responding to questions, and returning to interrupted work.

5.1.1 Planning Interface: Structuring and Isolating Work. Amelia begins by preparing and building her execution plan (Figure 5a–b). She pastes her tickets into the planning interface and attaches the relevant codebases and specification files from her issue-driven engineering workflow. She browses the available AGENTS.md files in ParallelPilot for suitable agent instructions. She then clicks **Analyze & Plan**.

The resulting planning view, shown in detail in Figure 6, presents task relationships on the left, an execution timeline in the center, and the ParallelPilot Assistant chatbot on the right. The analysis flags that **1a** pagination and **1b** CSV export both modify tickets.py, recommending that they run separately to avoid overlapping edits. This surprises Amelia, who has not worked with these files before. Separately, the graph identifies an output dependency: **2b** regression tests must verify the retry limits and failure behavior established during **2a** retry implementation.

The timeline proposes three agent windows and two waves: planning rounds that indicate task order, rather than requiring all tasks to start or finish together. **1a**, **1c**, and **2a** can proceed in parallel; **1b** and **2b** follow in their

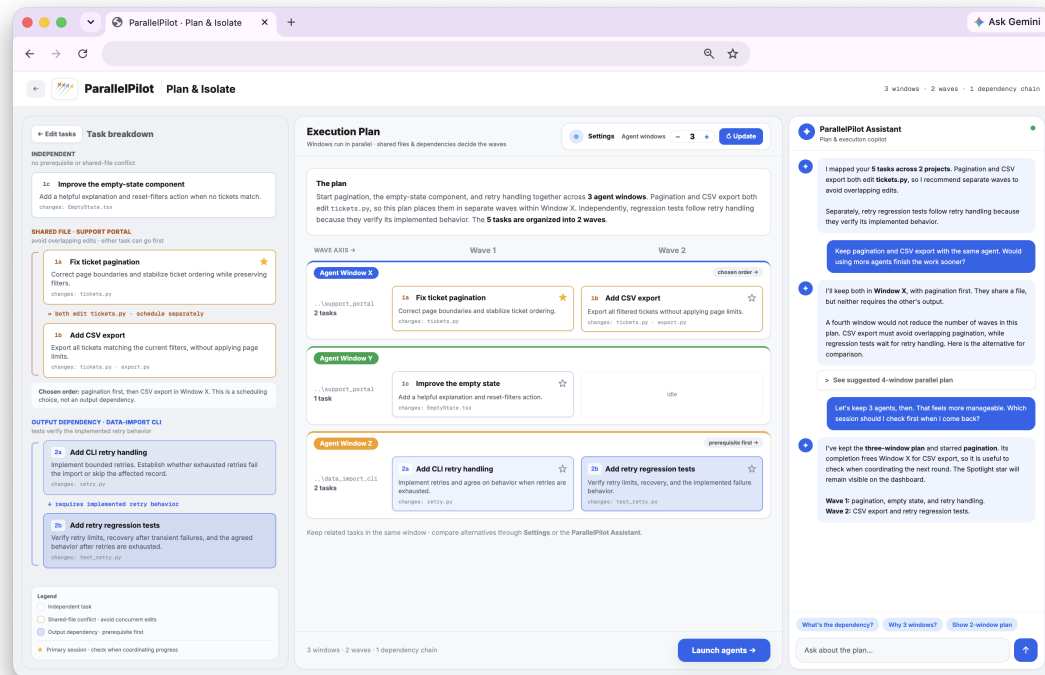


Fig. 6. ParallelPilot supports dependency-aware plan structuring and iteration. The task analysis (left) distinguishes independent work, shared-file conflicts, and output dependencies. The execution timeline (center) organizes five tasks across three agent windows and two waves, with pagination starred for follow-up. In the assistant panel (right), Amelia compares a four-window alternative and retains three windows to keep supervision manageable.

respective windows once the preceding work is reviewed and confirmed. A task in the second wave need not wait for unrelated work in other windows. Amelia asks the assistant, “Keep **1a** pagination and **1b** CSV export with the same agent. Would using more agents finish the work sooner?” The assistant offers a four-window alternative, explaining that another window would not reduce the number of waves under the current task breakdown and constraints. Amelia retains three agent windows to keep supervision manageable. The assistant marks **1a** with a Spotlight star because its completion frees that window for **1b**, making it a useful session to check when coordinating the next round. Satisfied, she accepts the plan and clicks `Launch Agents →` to open the per-agent setup instructions.

Externalizing dependencies and separating potentially conflicting work supports **Plan** and **Isolate**, while keeping model-generated recommendations open to user review and revision. Implementation details for dependency analysis, plan revision, and isolated session setup appear in Appendix A.1.

5.1.2 Run-Logger: Preserving Session Context. Amelia next launches the sessions with automatic logging, which preserves their activity for later review (Figure 5c). On the setup page, she finds a terminal command and a jumpstart prompt for each first-round agent. She runs each command to launch a Copilot CLI session, then pastes the corresponding prompt to link it to its assigned task and window ID. Three sessions begin working on **1a**, **1c**, and **2a**. Amelia keeps the terminals on her main monitor and the project tickets on her two extended monitors. As the agents work, the run-logger automatically preserves their activity and conversations in three layers: raw records, mid-length session memory, and brief tracking summaries. Amelia uses the tracking summaries to monitor progress at a glance. When a ticket needs closer attention, she opens its session memory to review changes, decisions, and reported results, cross-checking the CLI output as needed rather than reconstructing progress entirely from logs (Figure 5d). Raw records remain available in her local log folder for detailed inspection.

By combining detailed records with progressively shorter views, the run-logger supports monitoring and context recovery, instantiating the **Log** pillar. Implementation details for conversation capture, local persistence, and memory generation appear in Appendix A.2.

5.1.3 Ambient Dashboard: Monitoring and Triage. During execution, the ambient dashboard helps Amelia track progress, respond to agent questions, and review reported completion. Figure 7 provides a detailed view of these monitoring and triage interactions, introduced in Figure 5e–f. The dashboard opens automatically as Amelia starts the sessions, mapping the approved plan into *Done*, *Running Now*, and *Next* columns. Each card represents a task, with rows corresponding to agent windows and columns indicating execution stage. Active cards show a blue “running” status, files touched, and a one-line activity summary generated from logs. About a minute later, just as Amelia is halfway out of her chair for a bathroom break, the **2a** CLI retry card switches to a yellow “needs_attention” status and begins blinking. A thought bubble surfaces the agent’s question about whether to fail the import or skip a record after exhausting retries. Glad to catch it before stepping away, Amelia clicks `>_Terminal` and answers directly in the corresponding agent session. The card returns to blue “running” as work resumes. Amelia finds this familiar and comfortable: ParallelPilot provides an ambient coordination layer while she stays in control through direct interaction with the CLI agent. After her break, Amelia checks `# Session Memory` for the Spotlight-starred **1a** to catch up on its progress. Meanwhile, **1c** displays a green “completed” status and prompts her to inspect the result in the terminal. The **2a** CLI retry card also raises a follow-up question: “Should skipped records produce a non-zero CLI exit status?” These completion and attention cues appear together in Figure 7. Amelia reviews **1c**’s changes and confirms it as done, then returns to **2a**’s terminal to clarify the exit-status behavior and let it continue. Once **1a** completes, she reviews and confirms its result and starts **1b**. By the end of the run, Amelia feels she has balanced parallel progress

with manageable supervision. She has spent less effort coordinating sessions and reconstructing context, while the saved Markdown summaries preserve a record of changes and decisions for future reference.

These features instantiate **Observe** and **Triage**, enabling at-a-glance monitoring and targeted attention to sessions with prolonged inactivity, errors, input requests, or completion. Implementation details for dashboard updates, intervention detection, and completion verification appear in Appendix A.3.

5.2 System Implementation

ParallelPilot is a local-first web application that operates alongside existing coding CLIs. Its JavaScript/HTML interfaces use Preact and Tailwind CSS, with lightweight Node.js backends and PowerShell scripts for startup and session launch. Session records and generated artifacts are stored on the local filesystem, while model-assisted processing uses remote APIs. The planning interface uses chained calls to gpt-5.6-sol with medium reasoning, balancing dependency-analysis quality with latency, with medium reasoning effort to propose editable, dependency-aware assignments and generate launch instructions for agents working in isolated repository directories. The run-logger captures Copilot conversations through read-only SQLite polling and preserves raw records alongside session memory and tracking summaries generated by gpt-5-mini for speed. The ambient dashboard refreshes from local records approximately every 2.5 seconds and uses gpt-5.6-sol with medium reasoning to surface potential intervention needs. Across these components, developers retain direct access through their CLI sessions; ParallelPilot provides the planning, context preservation, and monitoring layer around them. Appendix A provides the full implementation details.

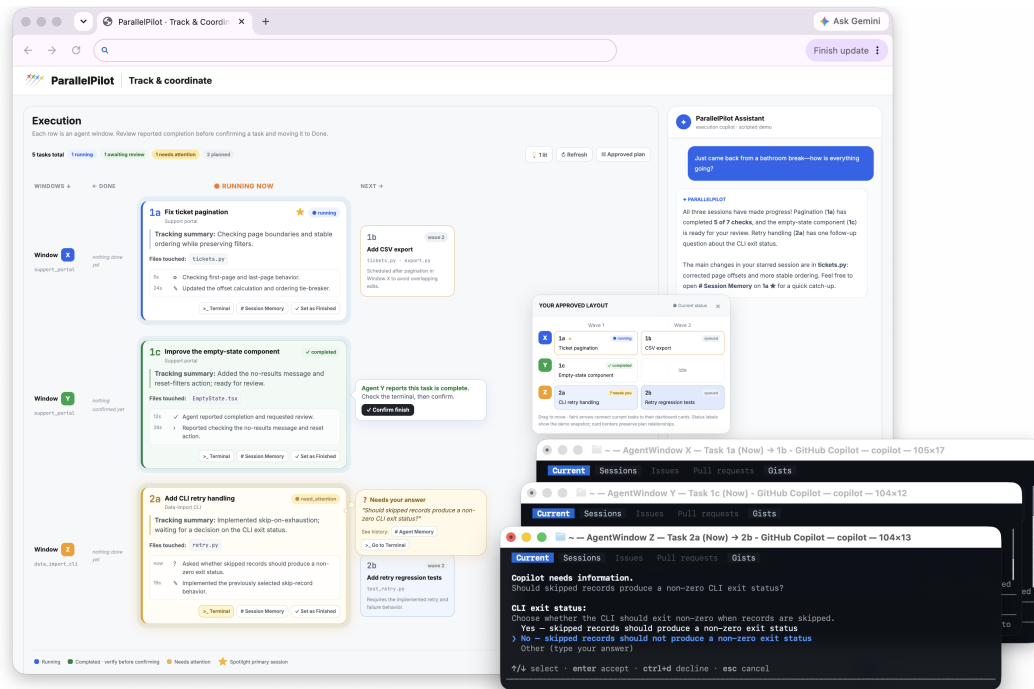


Fig. 7. ParallelPilot combines glanceable status, completion and attention cues, session memory, and terminal access to support monitoring and triage. After her bathroom break, Amelia catches up on parallel progress: **1a** remains running (blue), **1c** reports completion pending her review (green), and **2a** needs her input on the CLI exit status for skipped records (yellow). The dashboard surfaces completion and attention cues alongside session memory and a progress summary, while direct terminal access lets her review results and answer the agent in its CLI session.

6 ParallelPilot User Study

6.1 Overview

To explore how developers manage the demands of supervising parallel AI coding sessions, we conducted an 80-minute, in-person, counterbalanced within-subjects design probe with 16 developers. The study probes whether and how the PILOT-based design supports parallel AI coding in practice. We examine its effects on objective task outcomes and subjective user experience, as well as how it supports developers' coordination and monitoring across concurrent sessions. Finally, we investigate whether these forms of support translate into greater perceived control, more successful interventions, and increased trust. We operationalized these goals through three RQs and hypotheses (H):

- **RQ1 (Overall Effectiveness & User Experience):** Does ParallelPilot improve task completion^{H1a}, throughput^{H1b}, supervision capacity^{H1c}, and user experience^{H1d}?
- **RQ2 (Coordination & Monitoring Support):** How does ParallelPilot support dependency understanding^{H2a}, planning and task assignment^{H2b}, monitoring effort^{H2c}, and awareness of agent states and progress^{H2d}?
- **RQ3 (Control & Intervention Dynamics):** Does ParallelPilot support greater perceived control and intervention effectiveness^{H3a}, and trust and delegation confidence^{H3b}?

6.2 Participants

We recruited 16 participants (U1–U16) from an internal company research and engineering population through an email invitation. Recruitment targeted developers who regularly use AI coding assistants and have some experience with parallel AI coding workflows. Of the 16 participants (10 women, 6 men), 13 were in the 25–34 age range and three were in the 18–24 range. Fourteen participants reported substantial coding experience: 10 had 4–6 years and 4 had 7–10 years. Fifteen participants reported using AI coding assistants at least once per day, with 13 of 16 reporting use more than three times per day. All had prior parallel AI coding experience, but 15 rated themselves as Beginners or Somewhat Experienced, indicating that they were just getting started or still developing their approach.

Pre-study survey responses ($N = 16$) highlighted the challenges identified in the formative study. Eight participants reported difficulty decomposing tasks, and 14 reported experiences with overlapping or conflicting edits. Eleven found concurrent sessions difficult to track, 11 missed agent status changes, and 14 found session switching mentally taxing. Eleven spent effort reconstructing progress, 10 relied on session logs, 11 struggled to recall prior accomplishments, and 15 had difficulty reconstructing agents' decisions. Ten were unsure when to intervene. These responses were consistent with the coordination and monitoring needs that ParallelPilot was designed to address.

6.3 Study Design and Procedures

Each participant completed an in-person, 80-minute session, using ParallelPilot for one project and the no-ParallelPilot baseline for the other. Participants were randomly assigned to four groups, counterbalancing tool order and tool-project assignments. The projects, Sparkmatch and Kart, are described in the following subsection. This counterbalanced design let participants serve as their own control while mitigating order and task-difficulty effects. See Appendix B.1 for group procedures and Appendix C.2 for the session-order analysis.

Each 80-minute session began with a consent and setup briefing. Participants used a researcher-provided laptop and two external monitors, freely arranging windows across one, two, or three screens. See Appendix B.3 for apparatus and configuration details. Participants then proceeded through two counterbalanced coding blocks. Each of the two 30- to

35-minute coding blocks began with a 5-minute task introduction, in which the study administrator introduced the seed project, the tickets on a mock GitHub issue page, including requirements and acceptance criteria, and walked through the seed project codebase in VS Code. Participants had 20 minutes to complete the assigned coding tasks. In the ParallelPilot condition, participants additionally watched a 5-minute onboarding video introducing the PILOT framework and ParallelPilot, followed by a brief trial setup with the administrator. In the baseline condition, participants used GitHub Copilot CLI or Chat on the researcher-provided workstation. They could plan with the coding assistant, open multiple sessions, create worktrees, and maintain notes. Each block ended with a 5-minute post-condition survey assessing UX, coordination, and monitoring load, and perceived control. Sessions concluded with a post-study comparative survey and a 10-minute semi-structured interview reflecting on their experience across conditions. Participants also completed a pre-study survey covering demographics and prior experience with (parallel) AI coding. All participants provided informed consent to participate and to have the interview, screen, and code recorded for analysis. They were compensated 50 USD, and the study protocol was approved by the internal ethics committee. See Appendices F, G, H, and I for the pre-survey, post-condition survey, comparative survey, and interview questions.

6.4 Task Design

Participants worked on Sparkmatch, a minimally implemented dating application, and Kart, a partially implemented social shopping application. Each project included six tickets with explicit requirements and acceptance criteria on a mock GitHub issue page. Each project included two tasks requiring changes to the same file, one dependency chain requiring cross-task integration, and three other independently implementable tasks. One task in each project also required a product or policy decision. These features created opportunities to coordinate parallel work and resolve design choices with coding agents. To focus on implementation coordination and monitoring, neither condition required participants to manage pull requests, review code, or merge changes. Two developers who did not participate in the study pilot-tested the projects to assess comparable difficulty and completion times. Detailed task descriptions appear in Appendix B.2, and perceived-difficulty checks appear in Appendix ??.

6.5 Data Collection and Analysis

Our dataset comprised four surveys per participant (pre-study, two post-condition, and post-study comparison), interview recordings and transcripts, screen recordings, and system logs. We summarized survey responses descriptively and compared conditions using paired t -tests with Benjamini–Hochberg correction [40] for multiple comparisons. Results were considered statistically significant when unadjusted $p < .05$ and BH-adjusted $q < .05$. We used inductive affinity diagramming [11, 27] across transcripts and researcher notes, refining themes through feedback from all co-authors.

7 ParallelPilot User Study Results

7.1 RQ1 (Overall Effectiveness & User Experience): Does ParallelPilot improve task throughput, completion rates, and user satisfaction without introducing concurrent file conflicts?

Summary: Compared with the baseline, objective logs showed that ParallelPilot increased tickets completed per minute by 63% and total completion by 1.50 tickets. Participants scaled up parallel work by about 1 agent with ParallelPilot and anticipated supervising 1.44 more agents in future use than without it. Survey ratings showed significantly higher perceived efficiency (+1.69/7 points), better UX, and less perceived conflict. Table 1 provides evidence supporting H1a–H1d.

Table 1. Survey- and log-based results for RQ1, RQ2, and RQ3, comparing parallel AI coding with and without ParallelPilot ($N = 16$). Means are on a 1–7 scale unless otherwise noted. Paired p -values are two-tailed; BH q reports Benjamini–Hochberg-adjusted p -values across all hypothesis tests in the table. For both columns, * indicates $< .05$, ** indicates $< .01$, and *** indicates $< .005$. † indicates borderline BH-adjusted values $.05 \leq q < .06$. Green rows indicate measures that provide evidence supporting the corresponding RQ. Delta bars visualize the direction and approximate magnitude of the difference between conditions.

RQs / hypotheses		Measure / survey item	Baseline	ParallelPilot	Δ	p	BH q
RQ1: Overall Effectiveness & User Experience							
H1a	Ticket completion $\uparrow$	Tasks finished, out of 6 tickets	4.19	5.69	+1.50 tickets	.018*	.028*
	Full-completion participants $\uparrow$	Number of participants completing all 6/6 tickets	8/16	14/16	+6 users	–	–
H1b	Ticket Throughput $\uparrow$	Completed tickets per minute	0.272	0.445	+63%	.003***	.007**
	Task completion time $\downarrow$	Time on task, minutes	16.33	13.76	-2.57 min	.036*	.050 ⁺
	Finish time for dual completers $\downarrow$	Time-to-finish among dual completers, minutes	15.87	12.43	-3.44 min	.004***	.007**
H1c	Observed peak concurrency $\uparrow$	Maximum number of concurrent agents used during the session	2.31	3.25	+0.94 agents	.002***	.005**
	Perceived supervision capacity $\uparrow$	Num. of concurrent agents participants felt comfortable supervising in future	2.56	4.00	+1.44 agents	.001***	.002***
H1d	Perceived efficiency $\uparrow$	"I felt efficient working this way."	3.75	5.44	+1.69 / 7	<.001***	<.001***
	Reuse intent $\uparrow$	"I would want to work this way again on real tasks."	3.81	5.63	+1.81 / 7	<.001***	<.001***
	Ease of use $\uparrow$	"I found this setup easy to use."	4.44	5.56	+1.13 / 7	.014*	.015*
	Needs fit $\uparrow$	"The setup's capabilities met my needs for supervising multiple agents."	3.75	5.31	+1.56 / 7	<.001***	<.001***
	Workflow preference $\uparrow$	"For parallel AI coding, I prefer using ParallelPilot over my current setup."	–	14/16	–	–	–
RQ2: Coordination Attention – Task structuring (Plan, Isolate)							
H2a	Dependency understanding $\uparrow$	"I understood the dependencies between the pieces of work I assigned."	3.44	5.50	+2.06 / 7	<.001***	<.001***
	Perceived agent overlap $\downarrow$	"Agents ended up working on overlapping or conflicting parts of the code."	3.06	2.13	-0.94 / 7	.038*	.051 ⁺
	Isolation usefulness $\uparrow$	ParallelPilot-only usefulness: Dependency identification and task isolation	–	6.33 / 7	–	–	–
H2b	Plan clarity $\uparrow$	"I had a clear plan for how to break the work into parallel pieces."	3.63	5.00	+1.38 / 7	.007**	.012*
	Task-agent assignment fit $\uparrow$	"The way I assigned tasks to agents matched what each task actually needed."	3.81	4.56	+0.75 / 7	.013*	.034*
	Planning app usefulness $\uparrow$	Usefulness of ParallelPilot's plan and dependencies view	–	6.63 / 7	–	–	–
	Logs usefulness $\uparrow$	Usefulness of ParallelPilot's autosaving session logs / event history	–	5.09 / 7	–	–	–
RQ2: Monitoring Attention – Awareness & recall (Log, Observe, Triage)							
H2c	Tracking effort $\downarrow$	"Keeping track of what every agent was doing took a lot of mental effort."	4.63	2.75	-1.88 / 7	<.001***	<.001***
	Bookkeeping effort $\downarrow$	"I needed to do a lot of manual bookkeeping to stay on top of things."	4.63	2.50	-2.13 / 7	<.001***	<.001***
	Switching disruption $\downarrow$	"Switching between agents/sessions was disruptive to my focus."	4.50	2.69	-1.81 / 7	<.001***	.002***
	Context-switching $\downarrow$	"I felt like I was constantly context-switching rather than making progress."	3.81	2.25	-1.56 / 7	<.001***	.002***
H2d	Intervention cue awareness $\uparrow$	"I noticed when an agent finished, stalled, or needed my input."	3.63	6.19	+2.56 / 7	<.001***	<.001***
	Individual agent awareness $\uparrow$	"At any moment, I knew what each agent was working on."	4.00	5.06	+1.06 / 7	.044*	.057 ⁺
	Overall progress awareness $\uparrow$	"I had a clear overall picture of progress across all sessions at once."	3.44	6.06	+2.63 / 7	<.001***	<.001***
	Dashboard usefulness $\uparrow$	Usefulness of ParallelPilot's live monitoring dashboard	–	6.88 / 7	–	–	–
RQ3: Control & Intervention Dynamics							
H3a	Intervention success $\uparrow$	"When I intervened, it successfully (re)directed the agent."	3.94	4.13	+0.19 / 7	.603	.619
	Intervention confidence $\uparrow$	"I felt confident stopping, correcting, or steering an agent."	3.69	3.81	+0.13 / 7	.556	.646
	Perceived control $\uparrow$	"I felt in control of what the agents were doing."	3.94	4.31	+0.38 / 7	.414	.416
H3b	Output trust $\uparrow$	"I trusted the output produced in this block."	4.50	4.69	+0.19 / 7	.261	.416
	Delegation confidence $\uparrow$	"I felt confident delegating work to agents without watching them closely."	4.06	4.63	+0.56 / 7	.108	.128

7.1.1 [H1a & H1b] Higher completion and faster work from logs. Objective behavioral data show that ParallelPilot enabled participants to complete more work in less time. We used a lightweight implementation check, cross-referencing session recordings with the resulting code to verify that reported ticket completions corresponded to implemented changes. With ParallelPilot, participants completed an average of 1.50 more tickets than without it (5.69 vs. 4.19 of 6 tickets, $q = 0.028$). They also spent 2.57 fewer minutes on task (13.76 vs. 16.33 min) and achieved 63% higher throughput, completing 0.445 vs. 0.272 tickets per minute ($q = 0.007$). Completion rates increased substantially: 14 of 16 participants completed all six tickets with ParallelPilot, compared with 8 of 16 without it. Among participants who completed all

six tickets in both conditions, those using ParallelPilot finished 3.44 minutes faster on average (12.43 vs. 15.87 min, a reduction of roughly 22%). The split between coordination (planning and isolating) and later monitoring was essentially unchanged—25%/75% with ParallelPilot against 27%/73% without—so the gain came from compressing both phases rather than from shifting effort between them (see details in Figure 10 in Appendix C.4).

7.1.2 [H1c & H1d] Improved perceived efficiency, parallel supervision capacity, and UX. Self-reported measures closely aligned with the objective performance improvements. Participants rated their own efficiency 1.69 points higher with ParallelPilot, increasing from 3.75 to 5.44 on a 7-point scale ($q < 0.001$). They also reported less perceived overlap and conflict among their parallel agents' work, improving by 0.94 points (2.13 vs. 3.06). Participants reported that ParallelPilot was easier to use, better suited to their needs, and more useful for future parallel AI coding, with all corresponding comparisons statistically significant. Participants described ParallelPilot as “*much easier than [my setup]*” (U6), “*convenient and easier to proceed... very easy to use. I don't have to do a lot of manual work; it's more efficient*” (U1).

With ParallelPilot, participants used about 1 more agent in parallel than in the baseline (3.25 vs. 2.31 at peak use). Participants, who generally had limited parallel-coding experience, also anticipated comfortably supervising 1.44 more agents with ParallelPilot in future use ($p = 0.001$, $q = 0.002$), indicating increases in both observed concurrency and perceived supervision capacity. Overall, a strong majority (14/16) preferred ParallelPilot over baseline, with two exceptions: U9 found the experiences “*still somewhat similar*,” while U16 preferred the baseline, citing setup overhead and worrying that high-level abstractions reduced their control (“*scared of giving up control*”) and engagement with implementation details (“*[I ended up] not thinking as hard*”). We explore these perspectives further in Section 7.3.

7.2 RQ2 (Coordination & Monitoring Support): How does ParallelPilot support developers in coordinating parallel sessions, monitoring progress, and triaging agent states?

Summary: Survey ratings showed that ParallelPilot improved dependency understanding (+2.06/7 points) and final plan clarity (+1.38/7), reduced tracking effort (-1.88/7), and increased intervention-cue awareness (+2.56). Table 1 provides evidence supporting (H2a–H2d). Exploratory correlations linked coordination gains to objective productivity and monitoring gains to perceived efficiency.

7.2.1 [H2a & H2b] Improved coordination: clearer decomposition and dependency awareness. ParallelPilot substantially helped participants understand relationships among parallel tasks and build clear plans. Participants' ratings of their understanding of task dependencies increased from 3.44 to 5.50 ($q < 0.001$). Participants also reported less perceived overlap and conflict among agents' work from 3.06 to 2.13, thus rating the isolation feature as highly useful (6.33/7). Six of the eight participants who used ParallelPilot first reported later applying what they learned to the baseline condition, particularly by asking about file-sharing conflicts and sequencing dependencies between tasks. Building upon the beneficial decompositions, participants reported greater clarity in their overall plans, increasing from 3.63 to 5.00 ($q = 0.012$) and a better fit between tasks and assigned agents, increasing from 3.81 to 4.56. Participants then found the whole planning interface useful (6.63/7) and session logging useful for maintaining the information needed to coordinate work across agents and return to the task context later (5.09/7). U4 shared “*ParallelPilot is better at planning parallelization than me manual prompting [or... simply giving] Copilot a context dump [and asking it to execute].*”

7.2.2 [H2c & H2d] Improved monitoring: faster status awareness with less tracking effort. ParallelPilot also reduced the effort required to monitor ongoing agent activity while improving participants' awareness of what was happening across sessions. Participants reported substantially less effort spent tracking parallel work from 4.63 to 2.75

($q < 0.001$) and less bookkeeping to keep multiple agents on track from 4.63 to 2.50. U1 highlighted how separating the dashboard from individual CLI windows helped them “*keep track of overall progress better and manage multiple agents’ contexts [...], keep them separate but still stay engaged in both.*” With ParallelPilot, participants experienced less disruption when switching sessions (from 4.50 to 2.69) and less of a constant sense of context-switching rather than making progress (from 3.81 to 2.25). The resulting reduction in monitoring effort was accompanied by stronger situational awareness at both the local and global levels. Ratings of noticing when agents finished, stalled, or required input increased from 3.63 to 6.19 ($q < 0.001$). Awareness of each agent’s current work increased from 4.00 to 5.06, while awareness of overall progress increased from 3.44 to 6.06 ($q < 0.001$). U10 described seeing all agents’ completion notifications as “*the most satisfying UX feature,*” because it made their status immediately visible and allowed them to move on without unnecessary waiting. Participants therefore rated the live monitoring dashboard highly useful (6.88/7).

Exploratory correlations linked perceived coordination gains to gains in the objective efficiency (H1a ticket completion and H1b throughput), and perceived monitoring gains to gains in perceived efficiency (H1c). These associations are preliminary and do not establish component-specific effects; Appendix C.5 reports the analysis.

7.3 RQ3 (Control & Intervention Dynamics): Does ParallelPilot support greater perceived control, perceived success of interventions, and increased trust when supervising parallel AI coding sessions?

Summary: Despite improved intervention-cue awareness, intervention outcomes and perceived control did not significantly improve. Participants noted that ParallelPilot removed bookkeeping and tracking effort that had reinforced their sense of control, while ParallelPilot’s higher-level view changed how they engaged with implementation details.

[H3a & H3b] **Participants maintained similar levels of control-related outcomes, and why.** One of the clearest improvements in RQ2 was participants’ ability to detect cues of when agents needed intervention. However, we did not detect corresponding improvements in perceived control and intervention success. They changed little, from 3.94 to 4.31 and from 3.94 to 4.13, respectively. Confidence in their interventions, output trust, and delegation confidence also did not improve significantly. Consistent with these results, when asked which condition made output easier to verify, 9 participants reported no difference, 4 preferred ParallelPilot, and 3 preferred the baseline. Participants’ interview accounts suggest reasons why easier coordination and monitoring did not necessarily produce greater control or trust:

Coordination and monitoring support complement, rather than replace, evaluation and control. Knowing when to intervene does not necessarily mean knowing how to intervene. ParallelPilot focused on helping developers coordinate parallel tasks, follow progress, and identify agents needing attention. Participants’ accounts suggest that these benefits addressed a different need from evaluating an agent’s approach or correcting its implementation. For example, identifying a stalled agent provides a cue to inspect its work, but determining whether an active agent is pursuing the right approach may require implementation-level evidence beyond summary-oriented progress reports. Also, ParallelPilot’s plan-focused execution structure appeared to orient attention toward keeping work moving: identifying idle agents and detecting blockers. These are valuable coordination activities, while assessing whether an active agent is doing the right work can require a different kind of effort. An agent that appears busy or reports completion may still benefit from substantive inspection. When ParallelPilot surfaced a plan reference, U3 described “*a tendency to just keep going forward... less likely to ask follow-up questions in the terminal and deeply inspect.*” For trust and delegation confidence, participants noted that ParallelPilot primarily helped them supervise overall progress rather than evaluate the reliability of individual agents or the capabilities of the underlying AI models.

Coding assistants distance developers from the code, and ParallelPilot may now further shift them toward a managerial perspective. ParallelPilot’s existence as an abstraction layer helped participants manage plans, assignments, and progress across agents. The externalized tracking of ParallelPilot reduced bookkeeping effort, while participants described some manual tracking activities as opportunities to reinforce their understanding. U15 and U1 shared that, in their native workflow, every time they context-switch between sessions to check on progress, they reinforce their memory by reconstructing and scrutinizing what each agent is working on and how far it has progressed. Some participants described reconstructing session state less often because an automatically updated overview and markdown files were available. U10 described it as “*giving up my internal representation.*” U16 shared “[*with ParallelPilot,*] *I [ended up] not thinking as hard*” and U2 mentioned “*I feel like I’m getting dumber*” in the process. An automatically updated overview, therefore, supported awareness without necessarily preserving the detailed familiarity needed for intervention. Returning to agent terminals, inspecting changes, and asking targeted follow-up questions still required context reconstruction. Also, U5 described a shift in focus: “*With ParallelPilot, I will focus less on what each agent is doing but overall progress.*” U11 similarly drew an analogy to an engineer becoming a manager: their focus shifted “*from hands-on implementation [to...] now high-level goals and roadmap*”, leaving less detailed context for stepping in when an agent encountered problems.

8 Discussion

8.1 Learning from Human Management Practices

PILOT adapts concepts from human supervisory control literature [38] to motivate two areas of support: *coordination*, encompassing planning and teaching, and *monitoring*, encompassing monitoring and intervention. ParallelPilot operationalizes these concepts in an interface for parallel AI coding. In our evaluation, participants found that ParallelPilot helped address coordination and monitoring challenges, suggesting the value of adapting human management frameworks to agent supervision. Across both studies, six participants further used human management metaphors to describe their mental models in coordinating parallel agents. P6, a senior engineering manager, drew on their experience managing junior team members, describing how they used stand-up updates to assess whether someone’s approach aligned with what they considered an effective way forward. They applied a similar strategy to agent supervision, checking not only whether work was progressing, but also whether the agent was approaching the task appropriately. Senior researcher participants and PhD participants similarly described matching agents to tasks based on their “background” and skillsets, such as relevant specification files, much as they would assign research assistants based on their expertise. This suggests that prior supervisory experience can provide strategies for managing parallel AI coding.

Our evaluation suggests that interface support like ParallelPilot can help developers with varying levels of familiarity manage concurrent agents. However, tools are only one avenue for supporting this work. A complementary opportunity is to teach supervision strategies explicitly: how to structure parallel tasks, preserve shared context, establish checkpoints, recognize the need for intervention, and allocate attention across sessions. Whether training grounded in human management practices improves agent supervision remains an open empirical question. Future work could examine which practices transfer effectively, which require adaptation, and which introduce misleading expectations about agent capabilities or behavior. This inquiry also requires examining where the human management analogy breaks down. Human supervision involves social dynamics [13]: relationships, mutual accountability, and professional development. Productivity-oriented agent use therefore raises questions about which social dynamics remain relevant and how they shape developers’ expectations and oversight. Rather than treating PILOT as a complete solution, we see it as a starting point for studying how interface support and training can jointly develop effective agent-supervision practices.

8.2 The Abstraction Tension: Designing Future Human-AI Tools for Re-engagement

ParallelPilot illustrates the value of an abstraction layer over existing coding agents. Its planning interface externalizes task assignments and dependencies, while logs, status cards, and triage cues bring distributed activity into a shared view. Participants could notice agent needs more reliably, switch sessions less disruptively, and complete more work. At the same time, better visibility did not necessarily provide all of the context needed to evaluate outputs or intervene.

Participants' accounts suggest a tension between maintaining a high-level overview and staying engaged with implementation details. Externalized tracking reduced bookkeeping, and some developers described relying less on their own reconstruction of implementation progress and decisions. Knowing that an agent was finished, blocked, or requesting input provided useful situational awareness, but did not by itself establish whether its code was correct or its approach appropriate. For designers of similar tools, this suggests that abstraction should support not only an overview but also *re-engagement*: a clear path from summarized status to the code, decisions, and evidence needed for judgment.

This distinction matters when evaluating HCI productivity tools more broadly. Such tools often abstract complex workflows and automate selected activities to reduce human effort. Yet reductions in effort can change how users engage with the underlying work. This consideration resonates with questions of authenticity and authorship in co-creative tools, where reducing production effort raises questions about what constitutes meaningful human involvement. In visual analytics and clinical decision-support tools, reducing information-search and processing overhead should preserve users' ability to interpret evidence, question assumptions, and exercise judgment. Across these settings, the design challenge is not simply to minimize human effort, but to distinguish avoidable overhead from engagement that supports informed decisions. In parallel AI coding, meaningful involvement need not require writing every line, but it does require opportunities to understand, judge, and shape the resulting work.

Our findings point to early design insights for future coding or adjacent human-AI tools navigating this abstraction tension. First, tools should lower the cost of the inevitable re-engagement moment: when a developer needs to invest brainpower to reconstruct context. The moves in ParallelPilot are: one-click return to the original CLI and fallback resources; clear lead-in messages and timely notifications, reducing the need to reconstruct context and preventing delayed awareness; and persistent tracking and saving of history and files so disengaged work remains retrievable and promoting a sense of security. Second, and more fundamentally, why abstraction and disengagement happen in the first place is itself a design decision, not a fixed benefit or side effect of automation. A schema change and a syntax fix warrant different defaults, which is why the abstraction level should be specified per task rather than fixed per tool. But whichever level is chosen, it should remain legible to the developer, particularly during initial use, so they never have to guess how much the system is filtering out on their behalf. And it should be revisited over time: as a developer gains history with the system or proficiency with the tasks, the level of detail they actually need may shift, and whether that shift stays visible to them, rather than happening silently, is what makes this a genuine alignment question.

Scaffolding people out of as much of the work that does not require them as possible is not a new ambition in HCI. What our findings sharpen is the harder half of that ambition: calibrating how far disengagement can go before it costs judgment, aligning to users' needs and expectations as they shift over usage and time, and designing for re-engagement at the moments that actually call for it, rather than leaving developers to notice on their own.

9 Limitations and Future Work

Our work has four limitations. First, our formative study included 14 participants, primarily researchers and research interns. Although they reflect early adopters of parallel AI coding, future work should examine broader industry populations and development constraints, including speed-first MVP development and open-ended research engineering.

Second, ParallelPilot is a design probe of our framework, not an exhaustive exploration of its design space. Its dependency-aware plans and textual summaries offer one approach to coordination and monitoring, while its four intervention cues—completion, errors, questions, and being stuck—may not capture the full range of real-world intervention needs. Also, our findings depend on the specific model (gpt-5-mini for log summarization and gpt-5.6-sol with medium reasoning on all other tasks; access date: mid-August 2026) and the tool configurations used for ParallelPilot and GitHub Copilot. Future work should explore alternative ways to abstract, surface, and act on information, and examine their effects on attention, context recovery, and trust, as well as other model families.

Third, our ParallelPilot user study involved 16 participants in a counterbalanced within-subjects design using 20-minute blocks and six seed tasks. The short blocks may underestimate context-recovery costs and overestimate developers' ability to retain session state, limiting ecological validity. The fixed set of six seed tasks may also introduce a ceiling effect for completion: participants who completed all six tasks before the block ended could not demonstrate additional throughput. In addition, our evaluation focused on implementation throughput, so participants were not required to conduct systematic code review. Thus, we did not directly measure verification accuracy or implementation-level understanding. Our comparison evaluates ParallelPilot as an integrated package, combining assisted planning, isolation support, logging, monitoring, and triaging. It does not isolate the contributions of individual components or distinguish the benefits of additional automation from those of their interface presentation. Component-level comparisons are needed to establish these mechanisms. Future work should deploy ParallelPilot in the wild with larger, more diverse groups and repeated use to examine how familiarity shapes practice and how coordination extends to code review, reconciliation, and integration.

Finally, our studies capture rapidly evolving practices of parallel AI coding: the formative study took place in mid-June 2026 and the controlled study in mid-August 2026. Participants often developed strategies independently or through social learning. While our findings identify coordination challenges in current parallel AI coding, their specific manifestations may change as models, tools, and organizational norms evolve. Future work should examine how these challenges and developers' strategies develop over time.

10 Conclusion

Parallel AI coding turns developers' waiting time into active supervision of concurrent agents. Through formative interviews ($N = 14$), we identified PILOT: five supervisory practices for Planning, Isolating, Logging, Observing, and Triaging parallel sessions. Our evaluation of ParallelPilot ($N = 16$), a design probe instantiating PILOT, showed higher ticket throughput and perceived efficiency, with clearer plans, better dependency and progress awareness, and less tracking effort. Yet we detected no significant improvements in perceived control and self-reported intervention success. Participants' accounts suggest that externalizing supervision can ease coordination and monitoring without preserving the implementation understanding needed to intervene. Effective supervision support should therefore do more than reduce overhead: it should help developers re-engage with the code, decisions, and evidence needed to evaluate and redirect agent work.

References

- [1] Gagan Bansal, Jennifer Wortman Vaughan, Saleema Amershi, Eric Horvitz, Adam Fourney, Hussein Mozannar, Victor Dibia, and Daniel S. Weld. 2024. Challenges in Human-Agent Communication. arXiv:2412.10380 [cs.HC]. doi:10.48550/arXiv.2412.10380
- [2] Marcelo Vilas Boas, Gustavo Pinto, Edward Roberto Monteiro, Vinicius Fernandes Carida, and Danilo Ribeiro. 2026. One Developer Is All You Need: A Case Study of an AI-Augmented One-Person Squad in a Brownfield Enterprise. (May 2026). arXiv:2605.18461 [cs.SE]. doi:10.48550/arXiv.2605.18461
- [3] Marc Brysbaert. 2019. How many words do we read per minute? A review and meta-analysis of reading rate. *Journal of Memory and Language* 109 (2019), 104047. doi:10.1016/j.jml.2019.104047
- [4] Carlos Rafael Catalan, Lheane Marie Dizon, Patricia Nicole Monderin, and Emily Kuang. 2026. "I'm Not Reading All of That": Understanding Software Engineers' Level of Cognitive Engagement with Agentic Coding Assistants. In *CHI 2026 Workshop on Tools for Thought*. arXiv. arXiv:2603.14225 [cs.HC]. doi:10.48550/arXiv.2603.14225
- [5] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, and Markus Anderljung. 2024. Visibility into AI Agents. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)*. Association for Computing Machinery, New York, NY, USA, 958–973. doi:10.1145/3630106.3658948
- [6] Blake Crosley. 2026. Your Agent Writes Faster Than You Can Read. Section: AI & Technology. <https://blakecrosley.com/blog/cognitive-debt-agents>
- [7] Cursor. 2026. Faire doubles PR throughput with Cursor Cloud Agents · Cursor. <https://cursor.com/blog/faire>
- [8] Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, Xuantang Xiong, Lei Han, Zhiyuan Liu, and Maosong Sun. 2025. Multi-Agent Collaboration via Evolving Orchestration. arXiv:2505.19591 [cs.CL]. doi:10.48550/arXiv.2505.19591
- [9] Vaishali Dhanoa, Anton Wolter, Gabriela Molina León, Hans-Jörg Schulz, and Niklas Elmqvist. 2025. Agentic Visualization: Extracting Agent-based Design Patterns from Visualization Systems. arXiv:2505.19101 [cs.HC]. doi:10.1109/MCG.2025.3607741
- [10] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkang (Eric) Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 156, 15 pages. doi:10.1145/3706598.3713581
- [11] Jennifer Fereday and Eimear Muir-Cochrane. 2006. Demonstrating rigor using thematic analysis: A hybrid approach of inductive and deductive coding and theme development. *International journal of qualitative methods* 5, 1 (2006), 80–92.
- [12] Jiayi Geng and Graham Neubig. 2026. Effective Strategies for Asynchronous Software Engineering Agents. arXiv:2603.21489 [cs.CL]. doi:10.48550/arXiv.2603.21489
- [13] Katy Ilonka Gero, Tao Long, and Lydia B Chilton. 2023. Social Dynamics of AI Support in Creative Writing. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (Hamburg, Germany) (CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 245, 15 pages. doi:10.1145/3544548.3580782
- [14] Madeleine Grunde-McLaughlin, Hussein Mozannar, Maya Murad, Jingya Chen, Saleema Amershi, and Adam Fourney. 2026. Overseeing Agents Without Constant Oversight: Challenges and Opportunities. Version Number: 1. doi:10.48550/ARXIV.2602.16844
- [15] Ahmed E. Hassan, Gustavo A. Oliva, Dayi Lin, Boyuan Chen, and Zhen Ming (Jack) Jiang. 2026. Towards AI-Native Software Engineering (SE 3.0): A Vision and a Challenge Roadmap. *ACM Trans. Softw. Eng. Methodol.* 35, 9, Article 268 (Aug. 2026), 25 pages. doi:10.1145/3807901
- [16] Zoe Hitzig, Maxim Massenkoff, Eva Lyubich, Shaoyi Zhang, Ryan Heller, and Peter McCrory. 2026. *Agentic coding and persistent returns to expertise*. <https://www.anthropic.com/research/claude-code-expertise>
- [17] James Hollan, Edwin Hutchins, and David Kirsh. 2000. Distributed cognition: toward a new foundation for human-computer interaction research. *ACM Trans. Comput.-Hum. Interact.* 7, 2 (June 2000), 174–196. doi:10.1145/353485.353487
- [18] Ruanqianqian Huang, Avery Reyna, Sorin Lerner, Haijun Xia, and Brian Hempel. 2026. Professional Software Developers Don't Vibe, They Control: AI Agent Use for Coding in 2025. arXiv:2512.14012 [cs.SE]. doi:10.48550/arXiv.2512.14012
- [19] Edwin Hutchins and Tove Klausen. 1996. Distributed cognition in an airline cockpit. *Cognition and communication at work* 1 (1996), 15–34.
- [20] Jinglu Jiang, Alexander J. Karran, Constantinos K. Coursaris, Pierre-Majorique Léger, and Joerg Beringer. 2023. A Situation Awareness Perspective on Human-AI Interaction: Tensions and Opportunities. *International Journal of Human-Computer Interaction* 39, 9 (May 2023), 1789–1806. doi:10.1080/10447318.2022.2093863
- [21] Arun Joshi. 2026. Xai for coding agent failures: Transforming raw execution traces into actionable insights. *arXiv preprint arXiv:2603.05941* (2026).
- [22] Malik Khadar, Julia Cecil, Leon Van Der Neut, Nikola Banovic, Kevin Baum, Stevie Chancellor, Enrico Costanza, Motahhare Eslami, Anna Maria Feit, Susanne Gaube, Ujwal Gadiraju, and Harmanpreet Kaur. 2026. AI CHAOS! 2nd Workshop on the Challenges for Human Oversight of AI Systems. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*. Association for Computing Machinery, New York, NY, USA, Article 919, 6 pages. doi:10.1145/3772363.3778736
- [23] Brian Kitano, Evan Carlson, Bryan Russett, and Alex Kesling. 2026. Managing Multi-Agent Research Systems: A Dashboard for Human Oversight of Coordinating AI Agents. In *Proceedings of Human-centered Evaluation and Auditing of Language Models (HEAL@CHI'26)*. ACM, New York, NY, USA.
- [24] Banruo Liu, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, and Esha Choukse. 2026. Agentic Coding in the Wild: Characterizing GitHub Copilot Traces at Production Scale. arXiv:2608.00101 [cs.AI]. doi:10.48550/arXiv.2608.00101
- [25] Tao Long, Xuanming Zhang, Sitong Wang, Zhou Yu, and Lydia B. Chilton. 2025. DoubleAgents: Human-Agent Alignment in a Socially Embedded Workflow. In *2026 ACM Conference on Human-AI Complementarity and Alignment (HCOMP 2026)*. <https://arxiv.org/abs/2509.12626v3>

- [26] Jiaying Lu, Bo Pan, Jieyi Chen, Yingchaojie Feng, Jingyuan Hu, Yuchen Peng, and Wei Chen. 2024. AgentLens: Visual Analysis for Agent Behaviors in LLM-based Autonomous Systems. arXiv:2402.08995 [cs.HC] version: 1. doi:10.48550/arXiv.2402.08995
- [27] Andrés Lucero. 2015. Using affinity diagrams to evaluate interactive prototypes. In *IFIP conference on human-computer interaction*. Springer, 231–248.
- [28] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv. arXiv:2601.20404 [cs.SE]. doi:10.48550/arXiv.2601.20404
- [29] Gabrielle O’Brien, Alexis Parker, Nasir Eisty, and Jeffrey Carver. 2025. More code, less validation: Risk factors for over-reliance on AI coding tools among scientists. arXiv:2512.19644 [cs.SE] version: 1. doi:10.48550/arXiv.2512.19644
- [30] OpenAI. 2026. Research acceleration: The view inside OpenAI. <https://openai.com/index/research-acceleration-view-inside-openai/>
- [31] OpenAI. 2026. Understanding and counting tokens. <https://help.openai.com/en/articles/4936856-understanding-and-counting-tokens>
- [32] Bo Pan, Jiaying Lu, Ke Wang, Li Zheng, Zhen Wen, Yingchaojie Feng, Minfeng Zhu, and Wei Chen. 2024. AgentCoord: Visually Exploring Coordination Strategy for LLM-based Multi-Agent Collaboration. arXiv:2404.11943 [cs.HC]. doi:10.48550/arXiv.2404.11943
- [33] Max Pekarsky. 2024. You should keep a developer’s journal - Stack Overflow. <https://stackoverflow.blog/2024/12/24/you-should-keep-a-developer-s-journal/>
- [34] Zifan Peng and Mingchen Li. 2026. "What Did It Actually Do?": Understanding Risk Awareness and Traceability for Computer-Use Agents. arXiv:2603.28551 [cs.CR] <https://arxiv.org/abs/2603.28551>
- [35] Sarah Schömb, Yan Zhang, Jorge Goncalves, and Wafa Johal. 2026. From Conversation to Orchestration: HCI Challenges and Opportunities in Interactive Multi-Agent Systems. In *Proceedings of the 13th International Conference on Human-Agent Interaction (HAI '25)*. Association for Computing Machinery, New York, NY, USA, 158–168. doi:10.1145/3765766.3765795
- [36] Agnia Sergeyuk, Eric Huang, Dariia Karaeva, Anastasiia Serova, Yaroslav Golubev, and Iftekhar Ahmed. 2026. Evolving with AI: A Longitudinal Analysis of Developer Logs. arXiv:2601.10258 [cs.SE]. doi:10.1145/3744916.3787811
- [37] Thomas B. Sheridan. 1975. Considerations in Modeling the Human Supervisory Controller. *IFAC Proceedings Volumes* 8, 1, Part 3 (1975), 223–228. doi:10.1016/S1474-6670(17)67555-4
- [38] Thomas B. Sheridan. 2006. Supervisory Control. In *Handbook of Human Factors and Ergonomics*. John Wiley & Sons, Ltd, 1025–1052. Section: 38. doi:10.1002/0470048204.ch38
- [39] STOA. 2026. AI Factory: How One Developer Ships 72 Story Points/Day | STOA Docs. <https://docs.gostoa.dev/blog/how-we-built-ai-factory-ships-72-points-per-day>
- [40] David Thissen, Lynne Steinberg, and Daniel Kuang. 2002. Quick and easy implementation of the Benjamini-Hochberg procedure for controlling the false positive rate in multiple comparisons. *Journal of educational and behavioral statistics* 27, 1 (2002), 77–83.
- [41] Bogdan Vasilescu, Kelly Blincoe, Qi Xuan, Casey Casalnuovo, Daniela Damian, Premkumar Devanbu, and Vladimir Filkov. 2016. The sky is not the limit: multitasking across GitHub projects. In *Proceedings of the 38th International Conference on Software Engineering*. ACM, Austin Texas, 994–1005. doi:10.1145/2884781.2884875
- [42] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 50208–50232. <https://proceedings.mlr.press/v235/wang24h.html>
- [43] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: agent-computer interfaces enable automated software engineering. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 1601, 125 pages.
- [44] Xu Yang, Lunyiu Nie, Ethan Chandra, Stanislav Gannutin, Fangru Lin, and Swarat Chaudhuri. 2026. When Parallelism Pays Off: Cohesion-Aware Task Partitioning for Multi-Agent Coding. arXiv:2606.00953 [cs.LG]. doi:10.48550/arXiv.2606.00953
- [45] Asli Yardim, Raphael Serafini, Nadine Jost, Anna-Marie Orloff, Joshua Gabriel Speckels, and Alena Naiakshina. 2026. "The AI tool can't make it any worse." Investigating Developers' Security Behavior with AI Assistants in a Password Storage Study. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 1216, 33 pages. doi:10.1145/3772318.3791693
- [46] Jingheng Ye, Huiqi Zou, Simon Yu, and Weiyan Shi. 2026. Coding with "Enemy": Can Human Developers Detect AI Agent Sabotage? arXiv:2606.05647 [cs.AI] version: 1. doi:10.48550/arXiv.2606.05647
- [47] Wentao Zhang, Liang Zeng, Yuzhen Xiao, Yongcong Li, Ce Cui, Yilei Zhao, Rui Hu, Yang Liu, Yahui Zhou, and Bo An. 2026. AgentOrchestra: Orchestrating Multi-Agent Intelligence with the Tool-Environment-Agent (TEA) Protocol. arXiv:2506.12508 [cs.AI]. doi:10.48550/arXiv.2506.12508
- [48] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2026. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. arXiv:2606.30560 [cs.LG]. doi:10.48550/arXiv.2606.30560

Appendix

A ParallelPilot System Implementation Details

A.1 Planning Interface Implementation Details

Task dependency analysis and plan generation. The planning backend processes the user’s goal, ticket descriptions, specification files, and repository context through chained calls to gpt-5.6-sol with medium reasoning effort, balancing dependency-analysis quality with latency. These calls analyze task breakdowns, dependencies, and potential overlapping edits, distinguishing two constraint types — *output dependencies*, which require a particular execution order, and *shared-file conflicts*, which require non-overlapping work — before proposing agent assignments and execution rounds. The results populate a dependency graph, an editable drag-and-drop Gantt chart, and natural-language execution instructions. Execution rounds, presented as waves, indicate task order rather than global synchronization barriers: a task can proceed once its own prerequisites have been reviewed and confirmed, without waiting for the unrelated work in other windows.

Plan revision and session setup. Informed by our formative study, the default plan uses three parallel agents. Users can revise the plan directly or through the planning assistant, which uses the same model and reasoning effort as the planning backend. Model-generated dependency and assignment recommendations remain open to user review and revision. Clicking `Launch Agents →` opens the setup page and generates per-agent launch commands, jumpstart prompts, and steering files from the supplied specifications, including `AGENTS.md`, `CLAUDE.md`, and Copilot instructions. Each agent works in an isolated repository directory. The launch workflow associates sessions with their assigned tasks and window identifiers, connecting the approved plan to subsequent logging and monitoring.

A.2 Run-Logger Implementation Details

Capture and local persistence. The run-logger maintains three locally stored layers: raw, full-length event logs (`.json`); mid-length, cross-session Markdown memory (`.md`); and short UI-card summaries of key events (`.md`). Raw events and conversation records preserve structured agent activity, captured prompts and replies, and timestamps. The (`.json`) records are persisted using atomic file replacement. For Copilot, a background capture process polls the agent’s SQLite session store in read-only mode every two seconds. Conversation turns are additionally retained in append-only Markdown and JSONL archives that survive session resets. Logging and capture run in background Node.js processes, using the filesystem for persistence and read-only SQLite access for Copilot conversations.

Session memory and tracking summaries. When new conversation turns are detected, the system triggers memory and summary generation using gpt-5-mini. Mid-length session memory records context, decisions, progress, and key events, including intervention snippets and complete messages. Short tracking summaries are generated from the full log and provide one-line progress updates and key events for dashboard cards. Session memory is accessible through the agent detail view and can be reused as context for subsequent sessions or future ParallelPilot runs. Raw records remain available locally for detailed inspection, while the dashboard refreshes its view of the local records approximately every 2.5 seconds. The current Copilot capture integration relies on its SQLite session store. Logging and capture run in background Node.js processes, using the filesystem for persistence and read-only SQLite access for Copilot conversations. This design separates context preservation from the agent’s primary workflow: users do not need to interrupt an agent or ask it to produce a separate recap. We see that this lightweight approach can support other CLI agents through compatible logging integrations.

A.3 Ambient Dashboard Implementation Details

State updates and plan visualization. The dashboard maps the approved plan into task cards organized by agent window and execution stage: *Done*, *Running Now*, and *Next*. It polls the run-logger’s outputs and updates the interface through JavaScript approximately every 2.5 seconds, combining structured status and file-change events with generated tracking summaries. Planning-stage Spotlight selections remain visible to help developers prioritize sessions when coordinating subsequent work.

Intervention detection and completion review. A triage module uses gpt-5.6-sol with medium reasoning effort to identify four potential intervention types: prolonged inactivity, terminal errors, requests for user input, and completion messages. The first three trigger a needs_attention status, a glowing and blinking card, and a one-line explanation. Completion messages follow a separate path: the dashboard displays the agent’s reported completion and prompts the developer to verify the result before moving the task to *Done*. This separates an agent’s reported finish from the developer’s acceptance of its work; the completion cue is not itself a correctness check. Direct links to terminal sessions and Markdown history support inspection and intervention. Developers answer questions and review results in the underlying CLI sessions rather than through a replacement agent interface.

B User Study Method Details

B.1 Procedures Details



Fig. 8. Session procedure for the within-subjects study ($N = 16$, four groups of $n = 4$). Each 80-minute session opens with a 5-minute consent and setup briefing, then two counterbalanced coding blocks that cross tool condition (ParallelPilot vs. baseline without ParallelPilot) with project (Sparkmatch vs. Kart), and closes with a 10-minute post-study survey and interview.

B.2 Task Design Details

We designed two software projects to approximate realistic development experiences, each presented through a mock GitHub issue page containing six coding tickets with explicit requirements and completion criteria. Sparkmatch was an almost empty dating application. Its tasks covered compatibility-survey collection, a compatibility-matching algorithm, a ranked recommendation feed, keyboard-driven like/pass interactions and mutual-match detection, a match overlay and matches list, and profile viewing and editing. Kart was a partially implemented social shopping application with prebuilt authentication and friend-purchase lookup functionality. Its tasks covered profile display, login-history recording, login-gated checkout, catalog browsing and product recommendation with friend-purchase indicators like “N of your friends bought this,” shopping-cart operations, and password changes.

We aligned the projects on task counts and key coordination features: independently implementable tasks, dependency relationships requiring cross-task integration, one deliberately shared-file task pair, and one task requiring an explicit product or policy decision. The shared-file pairs were swipe interactions and match display in Sparkmatch, and checkout and cart display in Kart; the dependency chains connected preference management to matching in Sparkmatch and browsing history to recommendations in Kart. The decision-oriented tasks concerned matching policy and catalog presentation, respectively. These structures created opportunities to exercise separability, coordinate dependent work, and resolve design choices with coding agents in parallel, rather than through full AI automation. To focus on coordination and monitoring during implementation, participants in both conditions were asked to address the tickets without being required to manage pull requests, review code, or merge changes.

B.3 Apparatus Details

Participants used a researcher-provided laptop (Windows 11, 32GB RAM) connected to two 23-inch extended monitors, creating a three-screen workspace. They could freely arrange and move windows across screens as needed, simulating a realistic multi-monitor development environment. The stack included VS Code with GitHub Copilot (chat and CLI, default model gpt-5.6-sol medium reasoning, access date: mid-August 2026), two sandboxed repositories (Sparkmatch and Kart), and GitHub-style ticket lists with descriptions and epics, available in VS Code and as HTML. Each condition starts and concludes with a logging and cleaning command. In the ParallelPilot condition, the planning interface and the monitoring dashboard ran locally. Each session used a newly authenticated Copilot instance, isolated exclusively to that participant; no authentication state, chat history, or context memory persisted across sessions or participants, ensuring realistic behavior without carryover.

C User Study Result Details

C.1 Study Design Checks: Task Equivalence

Perceived difficulty ratings and supplementary equivalence checks on survey composites both point to broad comparability between tasks and sessions, though not perfect equivalence. In the post-study survey, 9 of 16 participants rated the two projects as similarly hard; the remaining 7 split on which was harder (3 Sparkmatch, 4 Kart).

We additionally computed five multi-item composites from the two post-condition surveys (one per task) and tested them for equivalence, as shown in Table 2. Each composite averaged its constituent items within a survey, and we paired each participant’s Kart and Sparkmatch scores for that composite across tool assignments. Under TOST (two one-sided tests) with a ± 1 -point margin on the 1–7 scale, four of the five composites met equivalence, with 90% CIs

falling entirely within the margin; the fifth, supervision-specific load, was inconclusive, with too wide a CI to establish either equivalence or a reliable difference.

Table 2. Paired task comparisons ($N = 16$). $\Delta = \text{Kart} - \text{Sparkmatch}$. Confidence intervals shown are 95%; TOST equivalence decisions use 90% intervals and a ± 1 margin.

Composite (item count)	Kart	Sparkmatch	Δ [95% CI]	TOST
Supervision-specific load (3)	3.54	3.33	+0.21 [−0.94, 1.35]	Inconclusive
Awareness, memory, and recall (3)	4.61	4.42	+0.19 [−0.72, 1.10]	Equivalent
Planning, decomposition, and delegation (5)	4.54	4.52	+0.01 [−0.82, 0.84]	Equivalent
Control, verification, and intervention (4)	3.76	4.31	−0.55 [−0.98, −0.12]	Equivalent
Usability and experience (5)	4.65	4.68	−0.03 [−1.07, 1.02]	Equivalent

C.2 Study Design Checks: Session-Order

We also compared first- and second-session composites using paired tests, as shown in Table 3. None of the five comparisons was statistically significant (all $p > .05$); observed standardized differences were small ($|d_z| \leq 0.29$). We therefore detected no session-order differences in these subjective measures, although the small sample does not rule out learning or carryover effects.

Table 3. Paired session-order comparisons ($N = 16$). $\Delta = \text{Session 2} - \text{Session 1}$; d_z is the paired standardized mean difference. All comparisons had $p > .05$.

Composite (item count)	Session 1	Session 2	Δ	d_z
Supervision-specific load (3)	3.60	3.27	−0.33	−0.16
Awareness, memory, and recall (3)	4.39	4.65	+0.26	0.16
Planning, decomposition, and delegation (5)	4.40	4.66	+0.26	0.17
Control, verification, and intervention (4)	3.90	4.17	+0.27	0.29
Usability and experience (5)	4.69	4.64	−0.05	−0.03

C.3 Participant Interactions with ParallelPilot’s Three Components

Figure 9 illustrates how participants used ParallelPilot’s three components within the study’s three-screen workspace (two external monitors and a laptop screen). With the **planning interface**, participants reviewed task dependencies and execution plans alongside project tickets and code (a). With the **run-logger**, they launched agent sessions with automatic activity capture while retaining their familiar VS Code and terminal interfaces (b). With the **ambient dashboard**, they kept an overview on the left external monitor while using the right monitor and laptop for individual agent sessions and code inspection (c–e). These arrangements supported monitoring parallel activity, investigating a potentially stalled session, and coordinating dependent follow-up work.

C.4 Task-Phase Time Allocation

Figure 10 shows that dual completers — the participants who completed both tasks in both conditions — maintain a similar split of task-phase time while saving time overall with ParallelPilot. Each of their tasks was coded into two phases: *Plan & Isolate*, from task start to the first launch, and *Launch + Monitoring*, from that launch to task completion. Time spent on optional testing is folded into Launch & Monitoring, since it falls inside that interval and only a minority

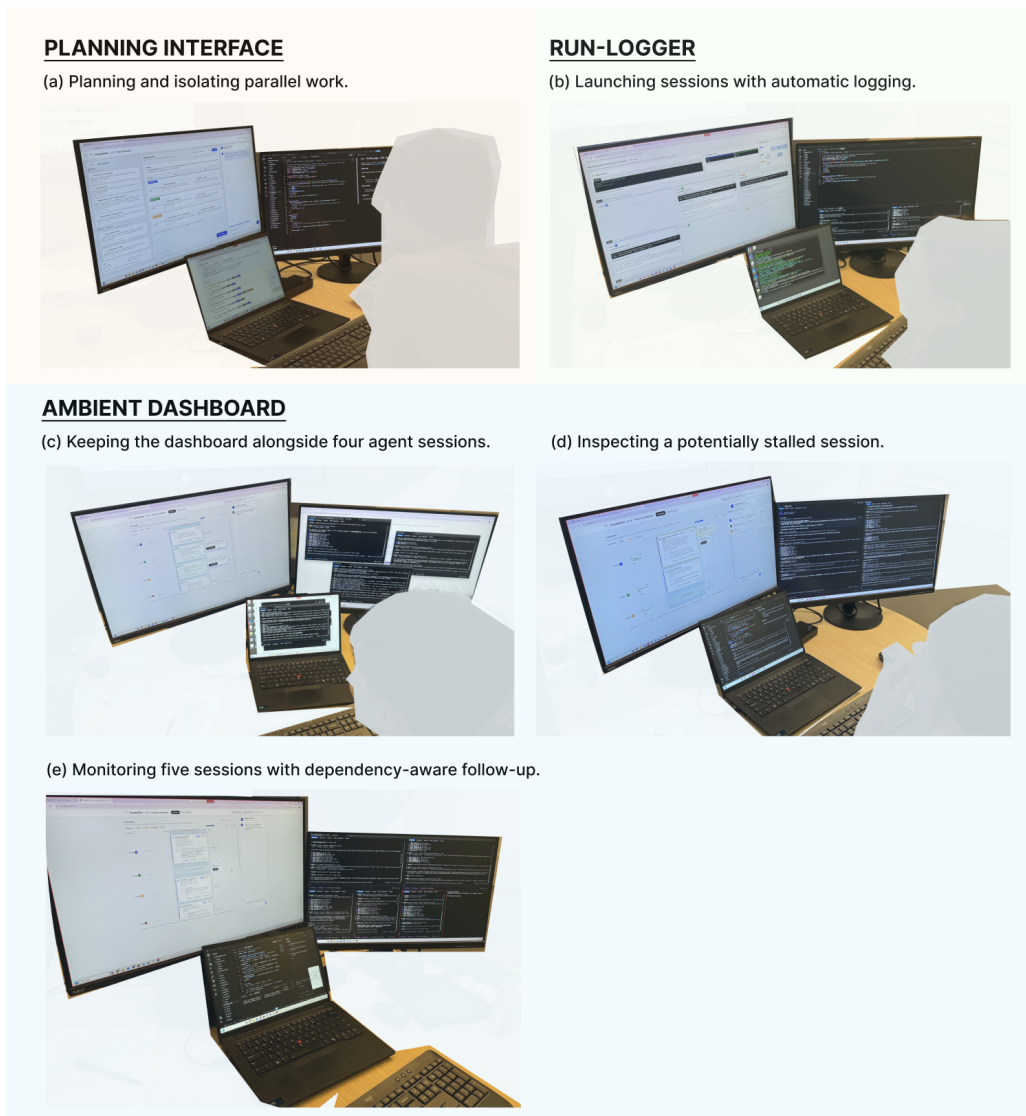


Fig. 9. **Participants' use of ParallelPilot's three components alongside existing coding tools.** (a) **Planning interface:** reviewing the execution plan and dependencies on the left external monitor, with code on the right monitor and project tickets on the laptop. (b) **Run-logger:** following launch instructions on the left monitor to start sessions with automatic logging, alongside VS Code terminals on the right monitor and a terminal on the laptop. (c–e) **Ambient dashboard:** keeping overall progress visible on the left monitor while (c) distributing four agent sessions across the right monitor and laptop; (d) inspecting a potentially stalled session on the laptop while two other sessions remain visible on the right monitor; and (e) tiling five agent sessions on the right monitor, with code open on the laptop and a dependent task reserved for a subsequent execution wave. Participants and surrounding room details have been masked for privacy and visual clarity.

of dual completers ran a separately coded testing phase. Panel A shows that the proportional split is nearly unchanged across conditions: Plan & Isolate accounts for 25% of task time with ParallelPilot versus 27% without, with Launch & Monitoring making up the remainder. Panel B shows that both phases are shorter in absolute terms with ParallelPilot,

by 1.27 and 2.17 minutes respectively, for a mean total of 12.43 minutes against 15.87 minutes. The tildes in Panel B mark means rather than observed task ends.

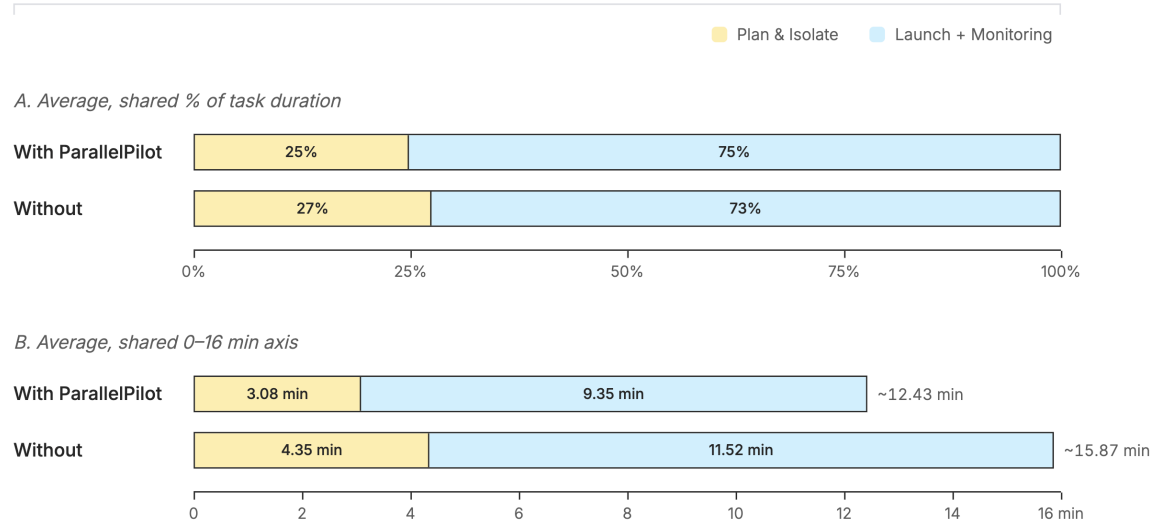


Fig. 10. Task-phase time allocation with and without ParallelPilot. Panel A gives the mean share of task duration spent on *Plan & Isolate* versus *Launch + Monitoring*; Panel B places the same means on a shared 0–16 min axis. Bars are means over the six participants who completed the task in both conditions.

C.5 [RQ2 → RQ1] Exploratory Associations Between Supervisory Support and Efficiency Gains

As shown in Figure 11, we conducted exploratory Spearman correlations to examine whether participant-level gains in *coordination* and *monitoring* were associated with gains in ticket completion (H1a), throughput (H1b), and perceived efficiency (H1c). We constructed composites by averaging all six coordination items (H2a, H2b) and all eight monitoring items (H2c, H2d). Gains were calculated as the with-ParallelPilot score minus the baseline score. All reported *p*-values are unadjusted.

Coordination-composite gains were associated with increases in objective efficiency metrics: tickets completed (H1a; $\rho = .55, p < 0.05$) and tickets completed per minute (H1b; $\rho = .62, p < 0.05$). Participants' accounts emphasized organizing work to avoid interference and reconciliation: U4 noted the clearer assignment and dependency isolation definitely reduced conflict risk when parallelizing, while U6 shared "[ParallelPilot's] workspace, worktree isolation helps agents [...] not collide with each other." These accounts suggest a possible link to objective productivity: avoiding conflicting work may reduce the rework needed to complete tickets.

Monitoring-composite gains, in comparison, were associated with perceived efficiency gains (H1c; $\rho = .62, p < 0.05$), but no significant associations were detected with objective productivity gains. Participants emphasized spending less effort checking progress: U4 described being able to "keep track of progress and manage multiple contexts in one place," while U1 explained, "[in my original workflow], I have to open many windows to track what finished, but since the dashboard already tracks it, I don't think I need to." U10 described seeing all three agents finish as "the most satisfying UX feature," because it made their status immediately visible and allowed them to move on without unnecessary waiting.

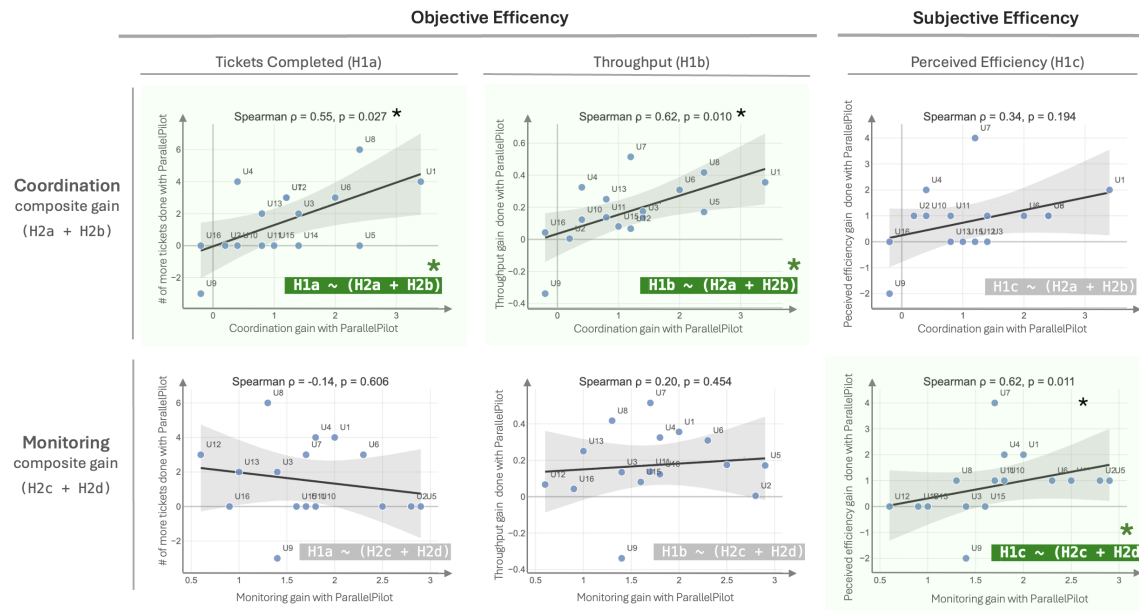


Fig. 11. Exploratory associations between participant-level gains in coordination (top row) and monitoring (bottom row) and gains in ticket completion, throughput, and self-rated efficiency ($N = 16$). Each point represents one participant. Annotations report Spearman correlations and unadjusted p -values; green panels and asterisks indicate $p < .05$. Black lines show descriptive OLS fits, with gray 95% confidence bands for the fitted means.

Together, these accounts suggest that monitoring support primarily improved the experience of supervising parallel work by reducing tracking effort and uncertainty about agent status, rather than directly increasing task output.

D Formative Study – Survey Questions

- (1) **Participant ID.** Please enter the participant ID provided by the researcher.
- (2) **Age** (optional). What is your age?
18–24, 25–34, 35–44, 45–54, 55–64, 65+
- (3) **Gender** (optional). What is your gender?
Woman, Man, Non-binary, Prefer not to say, Other
- (4) **Current Role.** What job title(s) best describe you currently? (You may list multiple roles, e.g., “Research Intern and PhD Student” if that applies.)
- (5) **Coding Experience.** How many years of coding experience do you have?
Less than 1 year, 1–3 years, 4–6 years, 7–10 years, More than 10 years, Other
- (6) **AI Coding Tools Used.** Which AI coding tools do you regularly use? (Select all that apply.)
CLI–Claude Code, CLI–Codex, CLI–Copilot, IDE–Claude, IDE–Copilot, IDE–Cursor, Web/Desktop–Codex, Web/Desktop–ChatGPT, Web/Desktop–Claude, Web/Desktop–Copilot, Other
- (7) **Frequency of AI Coding Assistant Usage (General).** How often do you use AI coding assistants in your coding workflow, including using just one agent at a time?
More than 3 times a day, 1–3 times per day, 1–3 times per week, 1–3 times per month, Fewer than once a month

- (8) **Frequency of Parallel AI Coding.** How often do you run multiple AI coding sessions in parallel?
More than 3 times a day, 1–3 times per day, 1–3 times per week, 1–3 times per month, Fewer than once a month
- (9) **Experience with Parallel AI Coding.** Approximately how many times have you used multiple AI coding sessions in parallel?
1–10 times, 10–30 times, More than 30 times
- (10) **Typical Number of Concurrent Sessions.** When working in parallel, how many AI coding sessions are typically active at once? (Please provide a range or a number.)
- (11) **Maximum Number of Concurrent Sessions.** What is the largest number of AI coding sessions you have had active at the same time? (Please provide a number.)
- (12) **Current Coding Workflow Distribution.** Thinking about your current coding practice, approximately what percentage of your work falls into each of the following categories? (Example: 40%, 40%, 20%. Then you can simply enter: 40, 40, 20. Please ensure your responses add up to 100%.)
- Coding without any AI tools: __%
 - Coding with a single AI coding tool/session at a time: __%
 - Coding with multiple AI coding sessions in parallel: __%
- (13) **Willingness to Share a Real Multi-Session Project.** During the interview, we will ask you to (screen)share and walk us through a real project where you used multiple AI coding sessions in parallel. To help prepare, please think of a project that you can show and discuss. During the interview, we will ask about:
- The background, context, and goals of the project.
 - How many AI sessions you used and how work was divided across sessions.
 - How you monitored, evaluated, and intervened in different sessions over time.
 - The benefits, challenges, surprises, or concerns you experienced while managing multiple sessions.
- Anything you choose to share will only be viewed by the research team. You may skip any project, repository, file, or detail you are not comfortable showing. Screen sharing will be used only to help the research team understand your workflow; we will not share your screen, code, or projects outside the research team. Any identifying or sensitive information (e.g., repository names, file names, proprietary code, or internal information) will be removed, anonymized, or blurred before appearing in notes or write-ups. You may stop screen sharing or skip any content at any time.
- Do you have a project in mind that you would be willing to walk us through during the interview?*
- Yes, I have a project in mind and would be willing to walk through it during the interview.

E Formative Study – Interview Protocols

For this study, an *AI coding session* was defined for participants as one ongoing interaction with an AI coding tool that has its own task context, for example a single Claude Code terminal, a Copilot or Cursor IDE chat, or one AI coding interface working on a specific task. We were especially interested in moments where more than one such session was active at the same time, even if the sessions were at different stages or required different levels of attention.

E.1 Warm-Up and Background

- Tell me a bit about your role and the kind of coding work you do day to day.
- When did you start using AI coding tools? What has your experience been like so far, and which tools or setup have you found most useful?

- What has your experience been like using multiple AI coding sessions at the same time?

E.2 Why Parallel

- *Pros*: Why do you run multiple AI coding sessions at the same time instead of one after another?
- *Cons*: What are the downsides or coordination costs of using multiple sessions at once?
- *Tradeoff*: When is using multiple sessions worth the effort, and when would you rather use just one session or none?

E.3 Task Walkthrough

Walk me through a recent, real setup where you had several sessions running, not a tidied-up or typical one.

E.3.1 Session Roster.

- What repo was this, and what were you trying to solve?
- *Session roster*: Can you give me a quick map of the sessions that were running? How many were there, and what was each one roughly working on?
- *Number rationale*: Why did you need that number of sessions?
- *Work division*: How did you decide how to divide the work across sessions?

E.3.2 Session Metadata.

- *Session purpose*: What was this session trying to do, and why did you make it a separate session rather than keeping it inside another session?
- *Work type*: Which best describes the work this session was doing?

E.3.3 Attention and Awareness.

- *When to check & reading strategy*: What do you actively monitor, and what do you usually skim or ignore? Did you read what the session was doing as it worked, or mostly inspect the result afterward?
- *Warning signals*: What kinds of things trigger you to step in, for example errors, weird output, drift, conflicts, or just a gut feeling?
- *Legibility of status*: Was anything important hard to see? Did this session have a clear signal for “everything is fine” versus “you need to inspect this carefully”?
- *Evaluation ability*: Did you know what a good result should look like, and how to evaluate whether the session was doing well? Were there parts where you felt unable to judge the quality of the work?

E.4 Closing Questions

- Tell me about the last time you intervened in an agent’s work.
- What’s the maximum number of concurrent sessions that feels useful, and what limits that number?
- If these agents were people, what management practices from managing humans carry over?

F ParallelPilot User Study – Pre-Survey

The pre-survey collected participants’ backgrounds, prior experience with parallel AI coding, and preferred screen configurations. Consent questions and name-collection fields are omitted below. Question numbers follow the original survey. All questions were required unless marked optional.

- 1717 (3) **Participant ID.** Please enter the participant ID provided by the researcher.
 1718 *Response:* Free text.
 1719
- 1720 (4) **Age (optional).** What is your age?
 1721 *Options:* 18–24; 25–34; 35–44; 45–54; 55–64; 65+.
 1722
- 1723 (5) **Gender (optional).** What is your gender?
 1724 *Options:* Woman; Man; Non-binary; Prefer not to say; Other.
 1725
- 1726 (6) **Coding experience.** How many years of coding experience do you have?
 1727 *Options:* Less than 1 year; 1–3 years; 4–6 years; 7–10 years; More than 10 years; Other.
 1728
- 1729 (7) **Frequency of AI coding assistant usage (general).** How often do you use AI coding assistants in your coding
 1730 workflow (including using just one agent at a time)?
 1731 *Options:* More than 3 times a day; 1–3 times per day; 1–3 times per week; 1–3 times per month; Fewer than once a
 1732 month; Other.
 1733 *Options:* 1–10 times; 10–30 times; More than 30 times; Other.
 1734
 1735
- 1736 (8) **Expertise with parallel AI coding.** Please select the group that best describes your parallel AI coding experiences.
 1737 • **Beginner:** I have only tried parallel AI coding a few times or am just getting started.
 1738 • **Somewhat experienced:** I have used parallel AI coding sessions occasionally and am still developing my
 1739 approach.
 1740 • **Experienced:** I regularly use parallel AI coding sessions and have established my own methods.
 1741 • **Expert:** I know how to set up parallel AI coding sessions, teach others, and have developed my own methods.
 1742
 1743
- 1744 (9) **Current experiences with parallel AI coding.** Reflecting on your current use of parallel AI coding sessions,
 1745 please indicate how much you agree or disagree with each statement below.
 1746 *Scale:* 1 = Strongly disagree; 4 = Neither agree nor disagree; 7 = Strongly agree.
 1747 (a) I find it difficult to break down coding goals into tasks for separate AI sessions.
 1748 (b) I struggle to determine which tasks can be run in parallel versus those that require sequential execution.
 1749 (c) I often realize too late that two agents worked on overlapping or conflicting code.
 1750 (d) It is challenging to keep track of what each agent is doing simultaneously.
 1751 (e) I often miss when an agent finishes, stalls, or becomes blocked.
 1752 (f) Switching between sessions or tabs to check status feels mentally taxing.
 1753 (g) I spend time figuring out which agent needs my attention next.
 1754 (h) I rely on reviewing raw terminals or logs to understand agent activity.
 1755 (i) After returning to a session, it is hard to recall what was previously accomplished.
 1756 (j) I find it difficult to reconstruct why an agent made certain decisions.
 1757 (k) I feel less in control when multiple agents are running at once.
 1758 (l) I am unsure when to intervene, redirect, or stop an agent.
 1759
 1760
- 1761 (10) **Preferred working setup.** What screen setup do you actively use for (coding and) parallel AI coding?
 1762 S = Laptop/Mac screen, H = Horizontal monitor, V = Vertical monitor. Only count screens actively used for coding
 1763 or coding-related tasks; do not count screens used only for non-coding activities (e.g., Teams, Spotify, YouTube).
 1764
 1765
 1766
 1767
 1768

Response: Multiple selection.

Options: S only; S + H; S + V; S + 2H; S + H + V; No preference—I am comfortable with any setup; Other.

(11) **Additional comments (optional).**

Response: Free text.

G ParallelPilot User Study – Post-Condition Survey

Participants completed this survey after each coding condition. The post-condition and comparative surveys were presented in the same form; their original question numbers are retained here.

Instructions. Please rate the following statements based on your experience during this AI coding session. Use a scale from 1 (Strongly disagree) to 7 (Strongly agree), unless otherwise specified. Please share your thoughts aloud while completing this survey.

(1) **Participant ID.**

Response: Free text.

(2) **Session task.** Select the project you worked on during this session.

Options: Sparkmatch; Kart.

(3) **Session condition.** Which condition did you experience during this session?

Options: Without ParallelPilot; With ParallelPilot.

(4) **Supervision-specific load.** Please rate the following statements about supervising agents/sessions.

Scale: 1 = Strongly disagree; 7 = Strongly agree.

- (a) Keeping track of what every agent was doing took a lot of mental effort.
- (b) Switching between agents/sessions was disruptive to my focus.
- (c) I felt like I was constantly context-switching rather than making progress.

(5) **Awareness, memory, and recall.**

Scale: 1 = Strongly disagree; 7 = Strongly agree.

- (a) At any moment, I knew what each agent was working on.
- (b) I noticed when an agent finished, stalled, or needed my input.
- (c) I had a clear overall picture of progress across all sessions at once.

(6) **Planning, decomposition, and delegation.**

Scale: 1 = Strongly disagree; 7 = Strongly agree.

- (a) I had a clear plan for how to break the work into parallel pieces.
- (b) I understood the dependencies between the pieces of work I assigned.
- (c) The way I assigned tasks to agents matched what each task actually needed.
- (d) Agents ended up working on overlapping or conflicting parts of the code.
- (e) I felt confident delegating work to agents without watching them closely.

(7) **Control, verification, and intervention.**

Scale: 1 = Strongly disagree; 7 = Strongly agree.

- 1821 (a) I felt in control of what the agents were doing.
- 1822 (b) When I intervened, it successfully (re)directed the agent.
- 1823 (c) I felt confident stopping, correcting, or steering an agent.
- 1824 (d) I trusted the output produced in this block.

1826 (8) **Usability and experience.**

1827 *Scale: 1 = Strongly disagree; 7 = Strongly agree.*

- 1829 (a) I found this setup easy to use.
- 1830 (b) The setup's capabilities met my needs for supervising multiple agents.
- 1831 (c) I needed to do a lot of manual bookkeeping to stay on top of things.
- 1832 (d) I felt efficient working this way.
- 1833 (e) I would want to work this way again on real tasks.

1834 (9) **Comfortable number of supervised agents/sessions.** Under this setup, how many agents or sessions do you feel you could comfortably supervise at the same time? Select all that apply.

1835 *Options: 1; 2; 3; 4; 5; 6; 7; 8+.*

1836 (10) **Session ID.**

1837 *Options: First Task Done; Second Task Done*

1838 **H ParallelPilot User Study – Final Comparative Survey**

1839 The final portion of the experience survey collected overall evaluations of ParallelPilot, feature usefulness ratings, and comparisons between the two conditions. Question 12 was required; Questions 13–16 were not marked required.

1840 (11) **Overall evaluation of ParallelPilot.** Please rate your agreement with each statement about your experience using ParallelPilot during this task.

1841 *Scale: 1 = Strongly disagree / Very negative; 4 = Neither agree nor disagree / Neutral; 7 = Strongly agree / Very positive. Item-specific anchors are given below where applicable.*

- 1842 (a) Overall, how would you rate your experience using ParallelPilot during this task? (1 = Very negative, 7 = Very positive)
- 1843 (b) Overall, how satisfied were you with ParallelPilot? (1 = Very dissatisfied, 7 = Very satisfied)
- 1844 (c) ParallelPilot helped me manage multiple AI coding sessions more effectively.
- 1845 (d) I would use a tool like ParallelPilot for future parallel AI coding tasks.
- 1846 (e) ParallelPilot improved my confidence in supervising multiple AI agents simultaneously.

1847 (12) **ParallelPilot usefulness.** Please rate the usefulness of the tool features.

1848 *Scale: 1 = Not at all useful; 4 = Neutral; 7 = Very useful; additional option: Didn't use. The survey instructions described the upper endpoint as "Extremely useful," while the response grid labeled it "Very useful."*

- 1849 (a) Plan / task decomposition view.
- 1850 (b) Workspace or branch isolation per agent.
- 1851 (c) Session logs / event history.
- 1852 (d) Live monitoring dashboard / status view.

1853 (13) **Setup/condition preference.** Please select one option for each aspect.

Options: Strongly prefer no-tool; Slightly prefer no-tool; No preference; Slightly prefer ParallelPilot; Strongly prefer ParallelPilot.

- (a) Overall preference.
- (b) Which setup made you feel more in control?
- (c) Which setup made it easier to stay aware of all agents?
- (d) Which setup made it easier to plan and split the work?
- (e) Which setup made it easier to verify the output?
- (f) Which setup felt less mentally taxing?
- (g) If we assume both tasks were equally hard, in which block do you think you produced better-quality work?
- (h) If we assume both tasks were equally hard, in which block do you think you produced more work?

- (14) **Task difficulty.** Was one task noticeably harder than the other?

Options: About equal; Yes, Kart is harder than Sparkmatch; Yes, Sparkmatch is harder than Kart.

- (15) **Additional comments.** Anything else you'd like us to know?

Response: Free text.

I ParallelPilot User Study – Interview Protocols

I.1 Post-task interview

The research protocol included the following open-ended post-task interview questions.

- (1) Compared with your normal workflow, what did the prototype make easier or harder?
- (2) Did the prototype help you remember what each session was doing? Give a concrete example.
- (3) Did the prototype help you notice risks, dependencies, conflicts, or places that needed verification?
- (4) Did the prototype change when you chose to intervene or let a session continue?
- (5) Which condition felt faster, safer, more controlled, or more mentally demanding?
- (6) Would you use something like this in your own workflow? What would need to change first?

I.2 Think-aloud prompts during the tasks

The protocol also included the following prompts for use during the coding sessions.

- (1) What are you trying to get this AI session to do right now?
- (2) What information are you using to decide whether this session is on track?
- (3) Which session needs your attention next, and why?
- (4) What would make you intervene, redirect, or stop this session?
- (5) If you came back after a few minutes away, what would you look at first to recover context?